COMPANION TO YOUR PLACEMENT PDF

Placement Knowledge Base

97 topics, one per page: the concept in depth, why it matters for your plan, key points, code, common mistakes, interview questions, practice problems and free resources. Tap a name below to jump to it; "Related" links at the end of each page jump between topics.

L0	L1	L2	L3	L4
Foundation	Core	Interview-ready	Advanced	Elite

Index

All 97 topics by category. Tap a topic to jump; ↑ Index on each topic brings you back.

Method 4

- Active recall 4
- Spaced repetition (1-3-7-21) 5
- The 20–40 minute rule 6
- Patterns, not problems 7
- Interleaving 8
- Speaking while you code 9
- The error log 10
- Upsolving 11
- Deep-work blocks (50/10) 12
- Sleep, exercise and energy 13
- Weekly review 14
- Day modes and the never-zero rule 15
- Using AI as a tutor 16
- The five difficulty levels 17

DSA 18

- Time and space complexity 18
- C++ and the STL 19
- Hashing and frequency maps 20
- Two pointers 21
- Sliding window 22
- Prefix sums 23
- Binary search 24
- Binary search on the answer 25
- Linked lists 26
- Recursion and backtracking 27
- Stacks and queues 28
- Monotonic stack 29
- Heaps and priority queues 30
- Greedy algorithms 31
- Interval problems 32
- Bit manipulation 33
- Binary trees 34
- Binary search trees 35
- BFS and graph basics 36
- DFS, components and cycles 37
- Dijkstra and shortest paths 38
- Topological sort 39
- Disjoint set union (Union-Find) 40
- Dynamic programming 41
- Knapsack and subset DP 42
- String DP: LCS and edit distance 43
- Bitmask DP 44
- DP on trees 45
- Tries 46
- Segment trees and Fenwick trees 47
- String algorithms: KMP, Z, hashing 48
- Competitive programming and contests 49

Core CS 50

- Processes and threads 50
- CPU scheduling 51
- Deadlock 52
- Synchronisation: mutex, semaphore, race conditions 53
- Memory management and virtual memory 54
- Normalisation and keys 55
- Transactions, ACID and isolation 56
- Indexing and B+ trees 57
- SQL for interviews 58
- OSI and TCP/IP models 59
- TCP vs UDP and the handshake 60
- HTTP, HTTPS and TLS 61
- What happens when you type a URL 62
- OOP: the four pillars 63
- Virtual functions and the vtable 64
- SOLID and design patterns 65

Design 66

- Low-level design (LLD) 66
- High-level system design (HLD) 67
- Caching and CDNs 68
- Load balancing, replication and sharding 69
- CAP theorem and consistency 70

Security 71

- OWASP Top 10 71
- SQL injection 72
- Cross-site scripting (XSS) and CSRF 73
- Broken access control and IDOR 74
- Server-side request forgery (SSRF) 75
- Hashing, encryption and passwords 76
- Threat modelling (STRIDE) 77
- Binary exploitation basics 78
- CTFs and security labs 79
- Bug bounty and responsible disclosure 80

Cloud 81

- AWS IAM and least privilege 81
- ABAC: attribute-based access control 82
- CloudTrail and CloudWatch alarms 83

TenantWall (your research project) 84
AWS core services and the SAA-C03 exam 85
AI 86
LLMs and RAG apps 86
Prompt injection and AI security 87
HFT 88
Probability for HFT interviews 88
Career 89
The interview loop (UMPIRE) 89
Online assessments (OAs) 90
Mock interviews 91
Behavioural interviews and STAR stories 92
The one-page resume 93
Projects that get shortlisted 94
Referrals and LinkedIn outreach 95
Cold emailing startups 96
The application pipeline 97
Preparing for a specific company 98
Offers, CTC and negotiation 99
Open source and GSoC 100

L0 · FOUNDATION METHOD

Active recall ↑ Index

> **WHY IT MATTERS FOR YOU**
> The single biggest learning multiplier. It's why the plan says 'solve before watching' and 'explain on a blank page'.

The concept

Retrieving information from memory strengthens it far more than re-reading it (the testing effect, Roediger & Karpicke 2006). Struggling to recall is the point; it feels slower but sticks.

In practice: close the video and solve, write a Core CS answer from memory before checking notes, and use flashcards that ask a question instead of showing a fact.

The Feynman technique: explain the concept in simple words on a blank page as if teaching a junior. Where you get vague is exactly what to re-learn.

Key points

- Notes in question form ('Why are threads cheaper?')
- Teach a friend or explain aloud
- Use the AI Feynman-checker prompt to find gaps

Common mistakes

× Highlighting and re-watching (feels productive, isn't)

Learn from

Roediger & Karpicke 2006 (PDF) (psychnet.wustl.edu) · Anki (apps.ankiweb.net)

Related: Spaced repetition (1-3-7-21) · The error log

L0 · FOUNDATION METHOD

Spaced repetition (1-3-7-21) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Without revision you forget most of a solved problem within weeks. This turns 550 problems into lasting skill.

The concept

Memory fades along a forgetting curve; each review at the right moment resets and flattens it. Spacing reviews out (1, 3, 7, 21 days) gives the most retention per minute.

For DSA: every problem you failed or needed a hint for goes on the list. On its due day, re-solve from a blank file in ≤ 10 minutes. Two clean passes in a row → retire it.

For Core CS facts: Anki does the scheduling automatically, 10 minutes a day.

Key points

- Revise before new problems each morning
- Keep revision to ~20% of DSA time
- Exam mode = revision only

Example

```
Google Sheets columns:
Date | Problem | Link | Pattern | Mistake | R1 | R3 | R7 | R21
R1: =A2+1   R3: =A2+3   R7: =A2+7   R21: =A2+21
Filter: any R column ⩽ TODAY() and not ticked
```

Common mistakes

× Re-reading old solutions instead of re-solving

Learn from

Anki manual (docs.ankiweb.net) · Gwern on spaced repetition (gwern.net)

Related: Active recall · The error log

L0 · FOUNDATION METHOD

The 20–40 minute rule ↑ Index

> **WHY IT MATTERS FOR YOU**
> Stops you wasting hours stuck, and stops you giving up too early. Both are common.

The concept

Easy/Medium: try for 20 minutes. Hard: 40 minutes. If stuck at the limit, read only the approach or hint (not code), then implement it yourself.

Stuck time is valuable up to a point: you are building the search for a pattern. Beyond it, you're just tired. The hint converts the struggle into learning.

Every problem you needed a hint for goes on the spaced-repetition list.

Key points

- Use the AI hint-only prompt for graded hints
- Read the editorial only until the key idea

Common mistakes

× Copying the full solution
× Spending 3 hours on one problem

Related: Spaced repetition (1-3-7-21) · The interview loop (UMPIRE) · Using AI as a tutor

L0 · FOUNDATION METHOD

Patterns, not problems ↑ Index

> **WHY IT MATTERS FOR YOU**
> 300 problems become about 25 reusable patterns, which is how strong candidates solve problems they've never seen.

The concept

After each solve, write one line: trigger → pattern. 'Sorted + pair sum → two pointers.' 'Minimum of the maximum → binary search on the answer.'

Before coding a new problem, scan your trigger list. If nothing fits, run the fallback checklist: can I sort it? hash it? binary search the answer? define states (DP)? model it as a graph?

Key points

- Keep the trigger list on one page
- Review it before every contest and interview

Common mistakes

× Memorising solutions instead of triggers

Learn from

NeetCode roadmap (neetcode.io)

Related: Two pointers · Sliding window · Binary search on the answer · Dynamic programming

L0 · FOUNDATION METHOD

Interleaving ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviews don't tell you the topic. Mixed practice trains you to identify the pattern, not just apply it.

The concept

Blocked practice (ten graph problems in a row) feels smooth because you already know the pattern. Interleaved practice (a graph, then a DP, then a string) forces you to choose, which is the real interview skill. Studies show interleaving improves later test performance even though it feels harder.

Use blocked practice when first learning a topic (Striver order), then interleave in revision days, contests and mocks.

Key points

- Sunday revision: random mix from the 1-3-7-21 list
- LeetCode 'pick one' or company-tagged lists are naturally interleaved

Common mistakes

× Interleaving before you know the basics of each topic

Related: Patterns, not problems · Spaced repetition (1-3-7-21)

L1 · CORE METHOD

Speaking while you code ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviewers score communication. Silence reads as 'stuck'. Tier-3 candidates practise this least.

The concept

Narrate: restate the problem, ask about constraints, say the brute force and its complexity, explain the better idea before coding, comment while coding ('this map stores value → index'), then dry-run aloud.

Practise one problem a day out loud, ideally recorded. Listen once a week for long silences and filler.

Key points

- Ask 'Shall I code this approach?' before coding
- Say complexity without being asked

Common mistakes

× Coding in silence for 10 minutes then explaining

Learn from

Pramp (free mock interviews) (pramp.com)

Related: The interview loop (UMPIRE) · Mock interviews

L0 · FOUNDATION METHOD

The error log ↑ Index

> **WHY IT MATTERS FOR YOU**
> Most wrong answers come from the same 10 mistakes. A log makes them visible so you stop repeating them.

The concept

Every wrong submission or failed interview question: one line with the problem, the mistake (off-by-one, int overflow, missed empty input, wrong pattern, misread question) and the fix.

Read the log before every contest, OA and interview. After a month, the top three mistake types become your personal pre-submit checklist.

Key points

- Tag mistakes by type to count them
- Pre-submit checklist: overflow? empty? n=1? duplicates? negative?

Common mistakes

× Logging without ever re-reading

Related: Spaced repetition (1-3-7-21) · Online assessments (OAs)

L1 · CORE METHOD

Upsolving ↑ Index

> **WHY IT MATTERS FOR YOU**
> The problem just past your level teaches the most; contests only show you where that is.

The concept

After a contest, take the first problem you couldn't solve. Try again for 30 minutes, then read the editorial idea, implement it yourself, and add it to the revision list.

One upsolved problem per contest beats two new contests without upsolving.

Key points

- Rating grows from upsolving, not from attempts

Common mistakes

× Skipping upsolving because the contest went badly

Learn from

Codeforces editorials (codeforces.com)

Related: Competitive programming and contests · The 20–40 minute rule

L0 · FOUNDATION METHOD

Deep-work blocks (50/10) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Hours at a desk don't measure progress; focused blocks do. Six blocks is a full-force day.

The concept

Work 50 minutes on one task with the phone in another room and notifications off; take a 10-minute break away from screens. Count blocks, not hours.

Put the hardest work (new DSA) in your sharpest hours, and revision, AWS and admin when tired. Decide each block's task the night before so you start instantly.

Key points

- Pomofocus timer
- Track blocks per day in your sheet
- 6 blocks = full force; 2 blocks = college day

Common mistakes

× Checking the phone 'just once' during a block

Learn from

Pomofocus (pomofocus.io) · Cal Newport, Deep Work (book) (calnewport.com)

Related: Day modes and the never-zero rule · Sleep, exercise and energy

L0 · FOUNDATION METHOD

Sleep, exercise and energy ↑ Index

> **WHY IT MATTERS FOR YOU**
> Memory consolidation happens during sleep. Cutting sleep to study more is a net loss.

The concept

Sleep 7–8 hours at a fixed time. Sleep after learning consolidates new memories; sleep deprivation hurts attention, problem-solving and recall the next day.

Exercise 20–30 minutes a day (walk, sport, gym) improves focus and mood. One rest half-day a week prevents burnout, which costs weeks.

Key points

- No all-nighters before contests or interviews
- Caffeine before 2 pm

Common mistakes

× Comparing yourself to LinkedIn posts at midnight

Learn from

Matthew Walker, Why We Sleep (book) (simonandschuster.com)

Related: Deep-work blocks (50/10) · Day modes and the never-zero rule

L0 · FOUNDATION METHOD

Weekly review ↑ Index

> **WHY IT MATTERS FOR YOU**
> Plans fail quietly. A 30-minute review each Sunday catches drift before it becomes a lost month.

The concept

Every Sunday: count problems, contests and blocks against the week's target; re-solve the week's failed problems; look at what slipped and why; set next week's three priorities.

Monthly (last Sunday, 1 hour): check the milestone checklist, take one timed mock on old topics, cut whatever isn't moving a number.

Key points

- Move unfinished items, never stack them on top
- Adjust quotas by ±1 problem, not ±5

Common mistakes

× Abandoning the plan after one bad week

Related: Day modes and the never-zero rule · Spaced repetition (1-3-7-21)

L0 · FOUNDATION METHOD

Day modes and the never-zero rule ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your weeks will differ: home, college, exams, bad days. A recipe for each removes the daily decision and keeps the streak.

The concept

Pick one mode each morning: Full force (6–8 blocks), College day (2 problems + AWS + flashcards), Exam mode (1 revision problem), Low energy (1 easy re-solve + read the error log), Sprint (mock + company set + applications).

Never zero: the streak is 'at least one problem touched'. A low-energy day that keeps the streak beats a perfect day followed by four days off.

Behind schedule? Cut the newest track (AI, then HFT), never DSA or revision, and never double tomorrow's quota.

Key points

- Plan tomorrow in 5 minutes tonight
- Exam mode 22 Mar – 7 Apr 2027

Common mistakes

× All-or-nothing thinking

Related: Deep-work blocks (50/10) · Weekly review

L0 · FOUNDATION METHOD

Using AI as a tutor ↑ Index

> **WHY IT MATTERS FOR YOU**
> Claude and Gemini can be a free tutor and interviewer, or a crutch that makes you feel productive while learning nothing. The rules decide which.

The concept

Use AI for: graded hints (one at a time), mock interviews, reviewing your accepted code, checking your Feynman explanations, generating flashcards, and resume critique.

Never paste a problem and ask for the solution before your 20/40-minute timebox. Interviewers spot borrowed understanding in two minutes of follow-ups.

Verify facts it gives you against docs; models can be confidently wrong, especially on exact numbers and APIs.

Key points

- Ask it to grade you 1–5 with one fix per area
- Use it to explain an editorial line you don't get

Example

```
Hint-only tutor prompt:
I'm solving [problem]. Don't give the solution or code. Give me one hint at a time,
the smallest nudge first. Wait for me to say 'next' before the next hint.
```

Common mistakes

- × Copying AI code into LeetCode
- × Trusting its complexity analysis without checking

Related: The 20–40 minute rule · Mock interviews

L0 · FOUNDATION METHOD

The five difficulty levels ↑ Index

> **WHY IT MATTERS FOR YOU**
> Tells you where to stop climbing on each topic, so you don't over-invest where your target jobs don't ask.

The concept

L0 Foundation: you can't do it yet; learn syntax, ideas and vocabulary. L1 Core: you can do it with notes (service-company level). L2 Interview-ready: cold, timed, out loud (product companies). L3 Advanced: you can handle the twist: Hards, contests, design trade-offs (Big Tech). L4 Elite: rare proof such as CF 1900+, CVEs, merged open-source work, HFT puzzles.

Your targets: DSA to L3, Core CS to L2, security to L3 (your edge), everything else to L1–L2 unless a specific job needs more.

Key points

- Climb in order
- Depth on one ladder beats breadth on five

Common mistakes

× Starting L4 topics before L2 is solid

Related: Dynamic programming · OWASP Top 10 · AWS core services and the SAA-C03 exam

L0 · FOUNDATION DSA

Time and space complexity ↑ Index

> **WHY IT MATTERS FOR YOU**
> Every interview answer ends with 'what's the complexity?'. It is also how you pick an approach before writing code: the input limit tells you which complexities will pass.

The concept

Big-O describes how running time or memory grows as the input size n grows, ignoring constants. A judge runs roughly 10^8 simple operations per second, so you can estimate before coding.

Read n from the constraints and work backwards: $n \le 10$ allows O(n!), $n \le 20$ allows $O(2^n)$, $n \le 500$ allows $O(n^3)$, $n \le 5000$ allows $O(n^2)$, $n \le 10^6$ needs O(n log n) or O(n), and $n \ge 10^9$ needs O(log n) or O(1) maths.

Space counts extra memory you allocate, including the recursion stack. A recursive DFS on a path of 10^5 nodes uses 10^5 stack frames, which can overflow.

Key points

- Nested loops over the same array → $O(n^2)$; halving each step → O(log n)
- Sorting is O(n log n); a hash map lookup is O(1) average, O(n) worst
- Amortised O(1): vector push_back doubles capacity occasionally
- Recursion: work per call × number of calls (draw the tree)
- Two pointers moving one way each → O(n) total, not $O(n^2)$

Common mistakes

× Forgetting the cost of string concatenation inside a loop
× Calling .size() or erase on the front of a vector inside a loop (erase is O(n))
× Saying O(1) for unordered_map without mentioning worst case when asked

Interview questions

☐ What is the complexity of your solution, time and space?
☐ Can you do better than $O(n^2)$?
☐ Why is building a heap O(n) and not O(n log n)?

Practise

Two Sum (leetcode.com) · Contains Duplicate (leetcode.com)

Learn from

Big-O cheat sheet (bigocheatsheet.com) · Striver A2Z sheet (takeuforward.org)

Related: C++ and the STL · Binary search

L0 · FOUNDATION DSA

C++ and the STL ↑ Index

> **WHY IT MATTERS FOR YOU**
> You chose C++ for DSA. In interviews and OAs, fluent STL saves 5–10 minutes per problem, and HFT firms test C++ more deeply than anyone.

The concept

Containers: vector (dynamic array), deque, list, set/map (red-black tree, ordered, O(log n)), unordered_set/unordered_map (hash table, O(1) average), priority_queue (binary heap, max by default), stack, queue.

Algorithms: sort, stable_sort, lower_bound/upper_bound (on sorted ranges), binary_search, reverse, accumulate, max_element, next_permutation, unique (with erase).

Idioms: auto and range-for, structured bindings (auto [a, b] = p), lambdas as comparators, emplace_back, fast I/O with ios::sync_with_stdio(false); cin.tie(nullptr).

Key points

- priority_queue<int, vector<int>, greater<int>> is a min-heap
- map[key] inserts a default if missing; use find or count to check
- lower_bound → first ≥ x; upper_bound → first > x
- Use long long when sums can pass $2 \cdot 10^9$
- Pass big containers by reference (const vector<int>&)

Code

```
vector<int> a = {5, 1, 4};
sort(a.begin(), a.end(), [](int x, int y) { return x > y; });
unordered_map<int,int> freq; for (int x : a) freq[x]++;
priority_queue<int, vector<int>, greater<int>> minh(a.begin(), a.end());
auto it = lower_bound(a.rbegin(), a.rend(), 4); // works on reversed order
```

Common mistakes

- × Iterator invalidation after erase/push_back
- × Comparator must be a strict weak ordering (use <, never <=)
- × Copying a vector into a function by value inside recursion

Interview questions

- ☐ Difference between map and unordered_map?
- ☐ How does vector grow?
- ☐ What is RAII?
- ☐ Why is unordered_map worst case O(n)?

Practise

Top K Frequent Elements (leetcode.com) · Group Anagrams (leetcode.com)

Learn from

takeuforward STL (takeuforward.org) · learncpp.com (learncpp.com) · cppreference (en.cppreference.com)

Related: Time and space complexity · OOP: the four pillars

L1 · CORE DSA

Hashing and frequency maps ↑ Index

> **WHY IT MATTERS FOR YOU**
> Turns many O(n²) 'have I seen this?' searches into O(n). It is the first optimisation interviewers expect.

The concept

A hash table stores keys in buckets chosen by a hash function. Average lookup, insert and delete are O(1); collisions make the worst case O(n).

Patterns: count frequencies, remember the index where a value was first seen, check complements (target − x), group by a canonical key (sorted string for anagrams).

Key points

- Two Sum: store value → index, check target − x before inserting x
- Anagram key: sorted string or 26-count array
- Use an array of 26/128 instead of a map when the alphabet is small

Common mistakes

× Inserting before checking (pairs an element with itself)

× Using unordered_map with pair keys (needs a custom hash)

Interview questions

☐ How does a hash map handle collisions?

☐ Why not always use a hash map?

Practise

Two Sum (leetcode.com) · Group Anagrams (leetcode.com) · Longest Consecutive Sequence (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Prefix sums · Sliding window

L1 · CORE DSA

Two pointers ↑ Index

> **WHY IT MATTERS FOR YOU**
> One of the five most frequent interview patterns. Turns pair searches in sorted data from O(n²) into O(n).

The concept

Keep two indices and move them based on a comparison. On a sorted array, if a[i] + a[j] is too small move i right; if too large move j left. Each pointer moves at most n times, so the total is O(n).

Variants: opposite ends (pair sum, container with most water), same direction fast/slow (remove duplicates, linked-list cycle), and fixing one element then running two pointers on the rest (3Sum, O(n²)).

Key points

- Needs sorted input or a monotone condition
- 3Sum: sort, fix i, two pointers on i+1..n−1, skip duplicates
- Partitioning (Dutch national flag) is three pointers

Code

```
sort(a.begin(), a.end());
int i = 0, j = n - 1;
while (i < j) {
    int s = a[i] + a[j];
    if (s == target) return {i, j};
    if (s < target) i++; else j--;
}
```

Common mistakes

- × Not skipping duplicates in 3Sum
- × Using it on unsorted data without sorting

Interview questions

- ☐ Why does moving the smaller side work in Container With Most Water?
- ☐ Prove two pointers doesn't miss a pair.

Practise

3Sum (leetcode.com) · Container With Most Water (leetcode.com) · Trapping Rain Water (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · NeetCode practice (neetcode.io)

Related: Sliding window · Binary search

L1 · CORE DSA

Sliding window ↑ Index

> **WHY IT MATTERS FOR YOU**
> Covers most 'longest/shortest substring or subarray such that…' questions, which appear in nearly every company's list.

The concept

Maintain a window [l, r]. Expand r each step; while the window breaks the rule, shrink l. Every index enters and leaves once, so it's O(n).

Fixed-size windows slide by adding a[r] and removing a[r−k]. Variable windows need the condition to be monotone: if a window is invalid, every bigger window containing it is invalid too.

'Exactly k' problems are often solved as atMost(k) − atMost(k−1).

Key points

- Track counts in a map or array for character windows
- Answer is updated after the shrink loop (longest) or inside it (shortest)
- Negative numbers break the monotone condition → use prefix sums instead

Code

```
int best = 0;
for (int l = 0, r = 0; r < n; r++) {
    add(a[r]);
    while (!valid()) remove(a[l++]);
    best = max(best, r - l + 1);
}
```

Common mistakes

- × Using it on sums with negative numbers
- × Forgetting to remove a[l] before l++

Interview questions

- ☐ Why is it O(n) when there's a nested while loop?
- ☐ How do you handle 'exactly k distinct'?

Practise

Longest Substring Without Repeating Characters (leetcode.com) · Minimum Window Substring (leetcode.com) · Longest Repeating Character Replacement (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Two pointers · Prefix sums · Hashing and frequency maps

L1 · CORE DSA

Prefix sums ↑ Index

> **WHY IT MATTERS FOR YOU**
> Range sums in O(1) and 'count subarrays with sum k' in O(n). Works with negative numbers where sliding window fails.

The concept

pre[i] = a[0] + … + a[i−1]. The sum of a[l..r] is pre[r+1] − pre[l].

To count subarrays summing to k: walk once, keep a map of how many times each prefix value has appeared; at each step add seen[pre − k]. Start with seen[0] = 1.

2D prefix sums give rectangle sums in O(1); difference arrays apply range updates in O(1) then rebuild.

Key points

- seen[0] = 1 handles subarrays starting at index 0
- Use long long for prefix arrays
- XOR prefix works the same way for XOR queries

Code

```
unordered_map<long long,int> seen{{0, 1}};
long long s = 0; int ways = 0;
for (int x : a) { s += x; ways += seen[s - k]; seen[s]++; }
```

Common mistakes

- × Off-by-one between pre[r+1] and pre[r]
- × Forgetting the initial seen[0] = 1

Interview questions

- ☐ Why doesn't a sliding window work for Subarray Sum Equals K?

Practise

Subarray Sum Equals K (leetcode.com) · Range Sum Query 2D Immutable (leetcode.com) · Continuous Subarray Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Hashing and frequency maps · Sliding window · Segment trees and Fenwick trees

L1 · CORE DSA

Binary search ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked at every level, and the bug-prone template interviewers watch closely. Step 4 of Striver's sheet.

The concept

On a sorted range, compare with the middle and discard half. O(log n).

Think of it as finding the boundary of a predicate that is false…false then true…true. Search for the first true. This one template covers lower_bound, first/last occurrence, rotated arrays and peaks.

Use mid = lo + (hi − lo) / 2 to avoid overflow.

Key points

- Rotated sorted array: one half is always sorted; check which
- Peak element: compare a[mid] with a[mid+1]
- Search a 2D sorted matrix as a flattened 1D array

Code

```
int lo = 0, hi = n - 1, ans = n;
while (lo ≤ hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] ≥ x) { ans = mid; hi = mid - 1; }
    else lo = mid + 1;
}
```

Common mistakes

- × Infinite loop from lo = mid with lo < hi
- × Wrong boundary returned (first vs last)

Interview questions

- ☐ Find the first and last position of an element.
- ☐ Search in a rotated sorted array with duplicates.

Practise

Search In Rotated Sorted Array (leetcode.com) · Find First And Last Position Of Element In Sorted Array (leetcode.com) · Find Peak Element (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search on the answer · Two pointers

L2 · INTERVIEW-READY DSA

Binary search on the answer ↑ Index

> **WHY IT MATTERS FOR YOU**
> A favourite Medium/Hard twist at Amazon, Google and in OAs. Recognising it is most of the work.

The concept

When the question asks for the minimum (or maximum) value X such that something is feasible, and feasibility is monotone in X, binary search over X and write an O(n) check(X).

Classic shapes: 'minimise the largest sum when splitting into k parts', 'minimum speed/capacity to finish in D days', 'maximise the minimum distance between cows'.

Key points

- Range of X: from the smallest possible answer to the largest
- check(X) is usually greedy
- Total O(n log range)

Code

```
long long lo = MIN_ANS, hi = MAX_ANS, ans = hi;
while (lo ⩽ hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) { ans = mid; hi = mid - 1; } else lo = mid + 1;
}
```

Common mistakes

× Wrong bounds (lo too high excludes the answer)

× Non-monotone check

Interview questions

☐ How do you know the answer space is monotone?

Practise

Koko Eating Bananas (leetcode.com) · Split Array Largest Sum (leetcode.com) · Capacity To Ship Packages Within D Days (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search · Greedy algorithms

L1 · CORE DSA

Linked lists ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pointer manipulation questions are common in first rounds and test careful coding. Step 6 of the sheet.

The concept

Each node holds a value and a pointer to the next. Insertion/deletion at a known node is O(1); access by index is O(n).

Core techniques: a dummy head node to avoid edge cases, fast/slow pointers (middle, cycle detection by Floyd's algorithm, cycle start), and iterative reversal with prev/curr/next.

Key points

- Reverse: three pointers, return prev
- Cycle start: after meeting, move one pointer to head, step both by 1
- Merge two sorted lists with a dummy node
- LRU cache = hash map + doubly linked list

Code

```
ListNode* prev = nullptr; ListNode* cur = head;
while (cur) { ListNode* nx = cur->next; cur->next = prev; prev = cur; cur = nx; }
return prev;
```

Common mistakes

- × Dereferencing null on fast→next→next
- × Losing the rest of the list before re-linking

Interview questions

- ☐ Detect and remove a cycle.
- ☐ Reverse nodes in k-groups.
- ☐ Design an LRU cache.

Practise

Reverse Linked List (leetcode.com) · Linked List Cycle Ii (leetcode.com) · Lru Cache (leetcode.com) · Reverse Nodes In K Group (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Two pointers

L1 · CORE DSA

Recursion and backtracking ↑ Index

> **WHY IT MATTERS FOR YOU**
> The base for trees, graphs and DP. Subsets, permutations and N-Queens are standard interview questions.

The concept

Recursion: solve a problem using smaller copies of itself; define the base case first, then trust the call.

Backtracking explores a decision tree: choose, recurse, un-choose. It's exhaustive, so complexity is the size of the tree (2^n for subsets, n! for permutations).

Prune early: stop a branch as soon as it cannot lead to a valid answer.

Key points

- Pick / not-pick for subsets
- Swap-based or used[] array for permutations
- Skip duplicates by sorting and checking i > start && a[i] == a[i−1]

Code

```
void go(int i, vector<int>& cur) {
    if (i == n) { res.push_back(cur); return; }
    cur.push_back(a[i]); go(i + 1, cur); cur.pop_back(); // pick
    go(i + 1, cur);                                     // skip
}
```

Common mistakes

- × Forgetting to un-choose
- × Copying vectors in every call

Interview questions

- ☐ Generate all valid parentheses.
- ☐ Solve N-Queens.
- ☐ What's the complexity of your backtracking?

Practise

Subsets II (leetcode.com) · Permutations (leetcode.com) · Combination Sum (leetcode.com) · N Queens (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming · Bitmask DP

L1 · CORE DSA

Stacks and queues ↑ Index

> **WHY IT MATTERS FOR YOU**
> Expression parsing, bracket matching and undo-style problems. Step 9 of the sheet, and the base for monotonic stacks.

The concept

A stack is last-in-first-out; a queue is first-in-first-out. Both give O(1) push and pop.

Use a stack when the most recent unfinished item matters (brackets, function calls, undo). Use a queue for processing in arrival order (BFS, scheduling).

Key points

- Valid parentheses with a stack of opening brackets
- Min-stack keeps a second stack of current minimums
- Queue using two stacks: amortised O(1)

Common mistakes

× Popping an empty stack

× Using std::stack when you need to iterate (use vector)

Interview questions

☐ Implement a queue with stacks.

☐ Design a min-stack.

Practise

Valid Parentheses (leetcode.com) · Min Stack (leetcode.com) · Evaluate Reverse Polish Notation (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Monotonic stack · BFS and graph basics

L2 · INTERVIEW-READY DSA

Monotonic stack ↑ Index

> **WHY IT MATTERS FOR YOU**
> Solves next-greater, stock span and largest-rectangle problems in O(n). A common Medium/Hard step-up.

The concept

Keep a stack whose values are always increasing (or decreasing). When a new element breaks the order, pop: each popped element has just found its next greater (or smaller) element.

Every element is pushed and popped once, so the total is O(n). Store indices, not values, so you can compute distances and widths.

Key points

- Largest rectangle in histogram: previous and next smaller
- Daily temperatures: next greater, answer = index distance
- Sum of subarray minimums: count contributions with previous/next smaller

Code

```
vector<int> nge(n, -1); stack<int> st;
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) { nge[st.top()] = a[i]; st.pop(); }
    st.push(i);
}
```

Common mistakes

× Strict vs non-strict comparison with duplicates (double counting)

Interview questions

☐ Largest rectangle in a histogram.
☐ Trapping rain water with a stack.

Practise

Daily Temperatures (leetcode.com) · Largest Rectangle In Histogram (leetcode.com) · Sum Of Subarray Minimums (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Stacks and queues

L2 · INTERVIEW-READY DSA

Heaps and priority queues ↑ Index

> **WHY IT MATTERS FOR YOU**
> Top-K, merging sorted lists, scheduling and Dijkstra all use a heap. Very common at Amazon.

The concept

A binary heap keeps the smallest (or largest) element on top. Push and pop are O(log n); peek is O(1); building from n items is O(n).

Top K largest: keep a min-heap of size k; pop when it grows past k. Streaming median: a max-heap for the lower half and a min-heap for the upper half, kept balanced.

Key points

- K-way merge: push the head of each list
- Use pair<int,int> to sort by priority then id
- Lazy deletion: skip stale entries when popped

Common mistakes

× Default priority_queue is a max-heap

× Sorting everything when you only need top k

Interview questions

☐ Find the median of a data stream.

☐ Why is heapify O(n)?

Practise

Kth Largest Element In An Array (leetcode.com) · Merge K Sorted Lists (leetcode.com) · Find Median From Data Stream (leetcode.com) · Task Scheduler (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dijkstra and shortest paths · Greedy algorithms

L2 · INTERVIEW-READY DSA

Greedy algorithms ↑ Index

> **WHY IT MATTERS FOR YOU**
> Short code, but you must justify it. Interviewers ask 'why does greedy work here?'.

The concept

Make the locally best choice at each step and never revisit it. It works only when an exchange argument holds: any optimal solution can be changed step by step into the greedy one without getting worse.

Most greedy solutions start by sorting: by end time (activity selection), by ratio (fractional knapsack), by deadline (job sequencing).

Key points

- Interval scheduling: sort by end, take if start ≥ last end
- Jump game: track the furthest reachable index
- If greedy fails on a small counterexample → it's DP

Common mistakes

× Assuming greedy works without testing a counterexample (0/1 knapsack)

Interview questions

☐ Prove your greedy choice is safe.

☐ When would greedy fail here?

Practise

Jump Game Ii (leetcode.com) · Non Overlapping Intervals (leetcode.com) · Gas Station (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Interval problems · Dynamic programming

L2 · INTERVIEW-READY DSA

Interval problems ↑ Index

> **WHY IT MATTERS FOR YOU**
> Merge intervals and meeting rooms are top-asked at Google, Microsoft and Amazon.

The concept

Sort by start (to merge) or by end (to choose the most non-overlapping). Then sweep once.

For 'how many overlap at once' (meeting rooms), sweep over events: +1 at start, −1 at end, and track the maximum; or use a min-heap of end times.

Key points

- Merge: if start ≤ last end, extend; else push new
- Insert interval: three phases (before, overlapping, after)

Common mistakes

× Touching intervals: decide whether [1,2] and [2,3] overlap

Interview questions

☐ Minimum meeting rooms needed.
☐ Merge overlapping intervals.

Practise

Merge Intervals (leetcode.com) · Insert Interval (leetcode.com) · Non Overlapping Intervals (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Greedy algorithms · Heaps and priority queues

L1 · CORE DSA

Bit manipulation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Quick wins in OAs (single number, power of two), and the base for bitmask DP.

The concept

x & 1 tests odd; x & (x−1) clears the lowest set bit; x & −x isolates it; a ^ a = 0, so XOR of all elements leaves the one that appears once.

A mask of n bits represents a subset of n items: loop mask from 0 to $2^n - 1$ to enumerate all subsets.

Key points

- __builtin_popcount(x) counts set bits
- Check bit i: (x >> i) & 1; set: x | (1 << i)
- Use 1LL << i for i ≥ 31

Common mistakes

× Operator precedence: write (x & 1) == 0 with brackets

Interview questions

☐ Find the two numbers that appear once.

☐ Count bits from 0 to n.

Practise

Single Number (leetcode.com) · Counting Bits (leetcode.com) · Subsets (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: bit manipulation (cp-algorithms.com)

Related: Bitmask DP · Tries

L2 · INTERVIEW-READY DSA

Binary trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> The largest single topic in interviews. Step 13–14 of the sheet.

The concept

Traversals: preorder, inorder, postorder (DFS) and level order (BFS with a queue). Know both recursive and iterative versions.

Most tree problems are 'return something from each subtree and combine': height, diameter, balanced check, max path sum. Write a function that returns the subtree answer and updates a global best.

Lowest common ancestor: if both sides return non-null, the current node is the LCA.

Key points

- Diameter = max over nodes of leftHeight + rightHeight
- Serialize with preorder + null markers
- Views (left/right/top/bottom) use BFS with column index

Code

```
int best = INT_MIN;
int gain(TreeNode* r) {
    if (!r) return 0;
    int L = max(0, gain(r->left)), R = max(0, gain(r->right));
    best = max(best, r->val + L + R);
    return r->val + max(L, R);
}
```

Common mistakes

- × Confusing 'path through node' with 'path returned to parent'
- × Recursion depth on skewed trees

Interview questions

- ☐ Maximum path sum.
- ☐ Serialize and deserialize a tree.
- ☐ Lowest common ancestor.

Practise

Binary Tree Maximum Path Sum (leetcode.com) · Lowest Common Ancestor Of A Binary Tree (leetcode.com) · Serialize And Deserialize Binary Tree (leetcode.com) · Diameter Of Binary Tree (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search trees · DP on trees · BFS and graph basics

L2 · INTERVIEW-READY DSA

Binary search trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> Common follow-up to tree questions; inorder-sorted property shows up everywhere.

The concept

Left subtree < node < right subtree. Inorder traversal gives sorted order. Search, insert and delete are O(h): O(log n) if balanced, O(n) if skewed.

Validate with a (low, high) range passed down, not just comparing with children.

Key points

- Kth smallest = kth node in inorder
- LCA in BST: walk down using values
- std::set/map are balanced BSTs (red-black)

Common mistakes

× Validating only parent-child pairs

Interview questions

☐ Validate a BST.

☐ Delete a node from a BST.

Practise

Validate Binary Search Tree (leetcode.com) · Kth Smallest Element In A Bst (leetcode.com) · Delete Node In A Bst (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary trees · Binary search

L1 · CORE DSA

BFS and graph basics ↑ Index

> **WHY IT MATTERS FOR YOU**
> Graphs are in almost every product-company loop. BFS gives shortest paths when every edge costs the same.

The concept

Store a graph as an adjacency list: vector<vector<int>> adj(n). Grids are implicit graphs with 4 neighbours.

BFS uses a queue and visits nodes in order of distance from the source, so the first time you reach a node is its shortest path. O(V + E).

Multi-source BFS starts with all sources in the queue (rotting oranges, nearest 0). 0-1 BFS uses a deque for edges of weight 0 or 1.

Key points

- Mark visited when pushing, not when popping
- Grid: dr/dc arrays for directions
- Bipartite check = BFS 2-colouring

Code

```
vector<int> dist(n, -1); queue<int> q; dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
}
```

Common mistakes

× Marking visited on pop → duplicates in queue

Interview questions

☐ Rotting oranges.
☐ Word ladder.
☐ Is the graph bipartite?

Practise

Rotting Oranges (leetcode.com) · Word Ladder (leetcode.com) · Is Graph Bipartite (leetcode.com) · Number Of Islands (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · CSES problem set (cses.fi)

Related: DFS, components and cycles · Dijkstra and shortest paths · Topological sort

L1 · CORE DSA

DFS, components and cycles ↑ Index

> **WHY IT MATTERS FOR YOU**
> Connected components, cycle detection and flood fill are standard graph questions.

The concept

DFS goes deep along one path before backtracking. Recursive or with an explicit stack. O(V + E).

Undirected cycle: you reach a visited node that isn't your parent. Directed cycle: you reach a node still on the current recursion path (keep an inStack array or 3 colours).

Count components by starting a DFS from every unvisited node.

Key points

- Flood fill on grids is DFS
- Directed cycle check = topo sort fails
- Bridges/articulation points use discovery and low times (L3)

Common mistakes

× Stack overflow on 10^6-node grids: use iterative DFS or BFS

Interview questions

☐ Detect a cycle in a directed graph.
☐ Number of islands.
☐ Clone a graph.

Practise

Number Of Islands (leetcode.com) · Clone Graph (leetcode.com) · Course Schedule (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · CSES problem set (cses.fi)

Related: BFS and graph basics · Topological sort · Disjoint set union (Union-Find)

L2 · INTERVIEW-READY DSA

Dijkstra and shortest paths ↑ Index

> **WHY IT MATTERS FOR YOU**
> The standard weighted shortest path. Asked at Google, Uber, Flipkart.

The concept

Dijkstra: a min-heap of (distance, node). Pop the closest, relax its edges. Works only with non-negative weights. O((V + E) log V).

Bellman-Ford relaxes all edges V−1 times, handles negative edges and detects negative cycles, O(VE). Floyd-Warshall gives all pairs in $O(V^3)$ for small V.

Key points

- Skip stale heap entries: if (d > dist[u]) continue
- Cheapest flights within K stops: BFS by level or Bellman-Ford with K rounds
- Path reconstruction with a parent array

Code

```
vector<long long> d(n, INF); d[s] = 0;
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq; pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du > d[u]) continue;
    for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) { d[v] = d[u] + w; pq.push({d[v], v}); }
}
```

Common mistakes

- × Using Dijkstra with negative edges
- × int overflow on distances

Interview questions

- ☐ Why does Dijkstra fail with negative weights?
- ☐ Network delay time.

Practise

Network Delay Time (leetcode.com) · Path With Minimum Effort (leetcode.com) · Cheapest Flights Within K Stops (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: Dijkstra (cp-algorithms.com)

Related: BFS and graph basics · Heaps and priority queues

L2 · INTERVIEW-READY DSA

Topological sort ↑ Index

> **WHY IT MATTERS FOR YOU**
> Course schedule and build-order questions; also how you detect cycles in directed graphs.

The concept

An order of a DAG's nodes where every edge goes forward. Kahn's algorithm: push all nodes with in-degree 0, pop, decrement neighbours, push any that reach 0. If you output fewer than V nodes, there's a cycle.

DFS version: add a node to the list after visiting all its children, then reverse.

Key points

- Alien dictionary: build edges from adjacent words
- Longest path in a DAG = DP in topo order

Common mistakes

× Not handling disconnected nodes

Interview questions

☐ Course schedule II.
☐ Alien dictionary.

Practise

Course Schedule Ii (leetcode.com) · Course Schedule (leetcode.com) · Parallel Courses (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: DFS, components and cycles · BFS and graph basics

L2 · INTERVIEW-READY DSA

Disjoint set union (Union-Find) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Fast connectivity for dynamic merging, Kruskal's MST and 'accounts merge'.

The concept

Each set has a root. find(x) follows parents to the root; union(a, b) links roots. With path compression and union by size, each operation is almost O(1) (inverse Ackermann).

Kruskal's minimum spanning tree: sort edges by weight, add an edge if its ends are in different sets.

Key points

- Count components = number of successful unions subtracted from n
- Redundant connection: the edge whose ends are already joined

Code

```
vector<int> p(n), sz(n, 1); iota(p.begin(), p.end(), 0);
function<int(int)> find = [&](int x) { return p[x] == x ? x : p[x] = find(p[x]); };
auto unite = [&](int a, int b) { a = find(a); b = find(b); if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b); p[b] = a; sz[a] += sz[b]; return true; };
```

Common mistakes

× Forgetting to call find before comparing

Interview questions

☐ Number of provinces.
☐ Accounts merge.
☐ Minimum spanning tree.

Practise

Number Of Provinces (leetcode.com) · Redundant Connection (leetcode.com) · Accounts Merge (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: DSU (cp-algorithms.com)

Related: DFS, components and cycles

L2 · INTERVIEW-READY DSA

Dynamic programming ↑ Index

> **WHY IT MATTERS FOR YOU**
> The topic that separates L2 from L3. Striver step 16 is the longest; product companies ask at least one DP.

The concept

DP = recursion + remembering answers to overlapping subproblems. Recipe: (1) define the state in words, e.g. dp[i] = best answer for the first i items; (2) write the transition from smaller states; (3) base cases; (4) the order to fill; (5) where the answer is.

Go memoised recursion → bottom-up table → space-optimised (keep only the last row). Interviewers love seeing all three.

Families: 1D (climbing stairs, house robber), grid paths, 0/1 and unbounded knapsack, subsequences (LIS, LCS, edit distance), partitions (palindrome partitioning, matrix chain), stocks (state machine), DP on trees, bitmask DP.

Key points

- Complexity = number of states × work per transition
- If greedy fails on a counterexample, think DP
- Print the solution by walking back through choices

Code

```
// 0/1 knapsack, space-optimised
vector<int> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int w = W; w ⩾ wt[i]; w--)
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
```

Common mistakes

- × Iterating capacity upwards in 0/1 knapsack (turns it unbounded)
- × Wrong base case for empty prefix

Interview questions

- ☐ Explain your state and transition.
- ☐ Can you reduce the space?
- ☐ Edit distance.
- ☐ Longest increasing subsequence in O(n log n).

Practise

House Robber (leetcode.com) · Coin Change (leetcode.com) · Longest Increasing Subsequence (leetcode.com) · Edit Distance (leetcode.com) · Partition Equal Subset Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · AtCoder Educational DP contest (atcoder.jp)

Related: Knapsack and subset DP · String DP: LCS and edit distance · Bitmask DP · DP on trees · Recursion and backtracking

L2 · INTERVIEW-READY DSA

Knapsack and subset DP ↑ Index

> **WHY IT MATTERS FOR YOU**
> Subset-sum, partition and coin-change are all knapsack in disguise.

The concept

0/1 knapsack: each item once; loop capacity downwards. Unbounded (coin change): items reusable; loop capacity upwards.

Subset sum to target T: boolean dp[t]. Partition equal subset: subset sum to total/2. Count of subsets with difference d: subset sum to (total + d)/2.

Key points

- Coin change (min coins): dp[x] = min(dp[x − c] + 1)
- Coin change II (count ways): loop coins outside, amounts inside

Common mistakes

× Order of loops changes combinations vs permutations

Interview questions

☐ Target sum.

☐ Partition equal subset sum.

Practise

Coin Change Ii (leetcode.com) · Target Sum (leetcode.com) · Partition Equal Subset Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming

L2 · INTERVIEW-READY DSA

String DP: LCS and edit distance ↑ Index

> **WHY IT MATTERS FOR YOU**
> Two-string comparison problems are a fixed pattern; once learned, a whole family falls.

The concept

dp[i][j] = answer for prefixes a[0..i) and b[0..j). If a[i−1] == b[j−1], extend the diagonal; otherwise take the best of dropping one character.

Edit distance: min of insert dp[i][j−1], delete dp[i−1][j], replace dp[i−1][j−1], each +1. Shortest common supersequence and longest palindromic subsequence (LCS of s and reverse(s)) follow.

Key points

- Space-optimise to two rows
- Distinct subsequences counts ways

Common mistakes

× Off-by-one between string index and dp index

Interview questions

☐ Edit distance.

☐ Longest common subsequence and print it.

Practise

Longest Common Subsequence (leetcode.com) · Edit Distance (leetcode.com) · Distinct Subsequences (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming · String algorithms: KMP, Z, hashing

L3 · ADVANCED DSA

Bitmask DP ↑ Index

> **WHY IT MATTERS FOR YOU**
> L3/L4 topic for hard rounds, OAs at top firms and HFT. Shows up when n ≤ 20.

The concept

The state includes a mask of which items are used: dp[mask] or dp[mask][last]. 2^n masks × n transitions → $O(2^n \cdot n)$ or $O(2^n \cdot n^2)$.

Travelling salesman: dp[mask][i] = shortest path visiting mask and ending at i.

Key points

- Iterate submasks: for (s = m; s; s = (s − 1) & m)
- Memory: 2^{20} × 20 ints ≈ 80 MB, watch limits

Common mistakes

× Using it when n is 40 (meet-in-the-middle instead)

Interview questions

☐ Shortest path visiting all nodes.

Practise

Shortest Path Visiting All Nodes (leetcode.com) · Partition To K Equal Sum Subsets (leetcode.com) · CSES: Elevator Rides (cses.fi)

Learn from

cp-algorithms: submask enumeration (cp-algorithms.com) · CSES problem set (cses.fi)

Related: Dynamic programming · Bit manipulation

L3 · ADVANCED DSA

DP on trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> Hard-round favourite: house robber III, max path sum, rerooting.

The concept

Compute dp for each node from its children in a post-order DFS. Often each node returns a small tuple (take, skip).

Rerooting: compute answers for one root, then move the root to each child in O(1) to get answers for all roots in O(n).

Key points

- House Robber III returns {rob, notRob}
- Sum of distances in tree is the classic rerooting problem

Common mistakes

× Recomputing subtrees ($O(n^2)$)

Interview questions

☐ House robber on a tree.

Practise

House Robber Iii (leetcode.com) · Sum Of Distances In Tree (leetcode.com) · CSES: Tree Distances I (cses.fi)

Learn from

USACO Guide: DP on trees (usaco.guide) · CSES problem set (cses.fi)

Related: Binary trees · Dynamic programming

L2 · INTERVIEW-READY DSA

Tries ↑ Index

> **WHY IT MATTERS FOR YOU**
> Autocomplete, word search and max-XOR problems. Last step of Striver's sheet.

The concept

A tree where each edge is a character; the path from the root spells a prefix. Insert and search are O(word length).

A bit trie stores numbers bit by bit from the top; to maximise XOR with x, greedily go to the opposite bit at each level.

Key points

- Node: array of 26 children + end flag (+ count for prefix counts)
- Word Search II: trie + DFS on the grid

Code

```
struct Node { Node* ch[26] = {}; bool end = false; };
void insert(Node* r, const string& w) {
    for (char c : w) { if (!r->ch[c - 'a']) r->ch[c - 'a'] = new Node(); r = r->ch[c - 'a']; }
    r->end = true;
}
```

Common mistakes

× Memory: 26 pointers per node; use array pools for big inputs

Interview questions

☐ Implement a trie.

☐ Maximum XOR of two numbers.

Practise

Implement Trie Prefix Tree (leetcode.com) · Word Search Ii (leetcode.com) · Maximum Xor Of Two Numbers In An Array (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Bit manipulation · String algorithms: KMP, Z, hashing

L3 · ADVANCED DSA

Segment trees and Fenwick trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> L3: range queries with updates. Needed for CP, CSES and harder OAs (Google, D. E. Shaw).

The concept

A segment tree stores an aggregate (sum, min, max) for each segment; query and point update are O(log n). With lazy propagation it supports range updates too.

A Fenwick tree (BIT) is shorter code for prefix sums with point updates, O(log n) each, using i & −i.

Key points

- Size the segment tree array 4n
- Count inversions with a BIT over compressed values

Code

```
// Fenwick, 1-indexed
vector<long long> bit(n + 1);
void add(int i, long long v) { for (; i ⩽ n; i += i & -i) bit[i] += v; }
long long sum(int i) { long long s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }
```

Common mistakes

× Forgetting to push lazy values down before recursing

Interview questions

☐ Range sum query, mutable.

☐ Count of smaller numbers after self.

Practise

Range Sum Query Mutable (leetcode.com) · Count Of Smaller Numbers After Self (leetcode.com) · CSES: Range Queries (cses.fi)

Learn from

cp-algorithms: segment tree (cp-algorithms.com) · cp-algorithms: Fenwick tree (cp-algorithms.com)

Related: Prefix sums

L3 · ADVANCED DSA

String algorithms: KMP, Z, hashing ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern matching in O(n + m). Appears in hard rounds and CP.

The concept

KMP builds a prefix function: pi[i] = length of the longest proper prefix of s[0..i] that is also a suffix. Matching never moves backwards in the text.

The Z-function z[i] = length of the longest substring starting at i that matches a prefix. Search pattern + '$' + text.

Rolling hash (Rabin-Karp) compares substrings in O(1) after O(n) preprocessing; use two moduli to reduce collisions.

Key points

- Shortest palindrome: KMP on s + '#' + reverse(s)
- Repeated substring pattern via prefix function

Common mistakes

× Single-modulus hashing can be broken by anti-hash tests

Interview questions

☐ Find the first occurrence of a pattern in O(n + m).

Practise

Find The Index Of The First Occurrence In A String (leetcode.com) · Shortest Palindrome (leetcode.com) · Repeated Substring Pattern (leetcode.com)

Learn from

cp-algorithms: prefix function (cp-algorithms.com) · cp-algorithms: Z-function (cp-algorithms.com) · cp-algorithms: string hashing (cp-algorithms.com)

Related: Tries · String DP: LCS and edit distance

L3 · ADVANCED DSA

Competitive programming and contests ↑ Index

> **WHY IT MATTERS FOR YOU**
> Ratings are a filter recruiters understand, contests train speed under pressure (exactly what OAs test), and HFT firms hire heavily from strong CP profiles.

The concept

Do one LeetCode weekly contest and one Codeforces Div 2/3 round a week. After each, upsolve the first problem you couldn't solve.

The CP31 sheet gives 31 problems per rating band (800 → 1900): climb in order. CSES gives 300 classic problems by topic.

Targets from your plan: LC 1800+ or CF 1400+ by mid-2027 for Big Tech; CF 1600–1900+ for HFT.

Key points

- Read all problems first; solve in order of confidence
- Stress-test with a brute force on random inputs
- Keep a contest log: problem, idea, mistake

Common mistakes

× Grinding new contests without upsolving
× Chasing rating over interview skills before L2 is done

Interview questions

☐ (Interviews rarely ask about CP; your rating speaks on the resume)

Practise

Codeforces (codeforces.com) · LeetCode contests (leetcode.com) · CSES problem set (cses.fi) · AtCoder (atcoder.jp)

Learn from

USACO Guide (usaco.guide) · cp-algorithms (cp-algorithms.com) · clist.by contest calendar (clist.by)

Related: Online assessments (OAs) · Time and space complexity

L1 · CORE CORE CS

Processes and threads ↑ Index

> **WHY IT MATTERS FOR YOU**
> The most-asked OS question in every technical round, product or service company.

The concept

A process is a running program with its own address space (code, data, heap, stack), file descriptors and a PCB (process control block) in the kernel.

A thread is a unit of execution inside a process. Threads share the process's code, data, heap and files, but each has its own stack, registers and program counter.

Threads are cheaper because creating one and switching between threads of the same process don't need a new address space or a TLB flush. The cost of sharing is that one thread's bug (or a missing lock) can corrupt the whole process.

Key points

- Process states: new, ready, running, waiting, terminated
- fork() copies the process (copy-on-write); exec() replaces its image
- User-level vs kernel-level threads
- Zombie: finished but parent hasn't called wait(); orphan: parent died first

Common mistakes

× Saying threads share the stack (they don't)

Interview questions

☐ Process vs thread, and why threads are cheaper?
☐ What happens on a context switch?
☐ What is a zombie process?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · GfG: process vs thread (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com)

Related: CPU scheduling · Synchronisation: mutex, semaphore, race conditions · Memory management and virtual memory

L1 · CORE CORE CS

CPU scheduling ↑ Index

> **WHY IT MATTERS FOR YOU**
> Numerical questions in OAs (compute waiting time) and conceptual ones in interviews.

The concept

The scheduler picks which ready process runs next. Goals: high CPU use and throughput, low waiting, turnaround and response time, fairness.

FCFS (convoy effect), SJF/SRTF (optimal average waiting, needs burst estimates, can starve long jobs), priority (starvation fixed by ageing), round robin (time quantum; good response time), multilevel feedback queues (what real systems approximate). Linux uses CFS, which tracks virtual runtime in a red-black tree.

Key points

- Turnaround = completion − arrival; waiting = turnaround − burst
- Preemptive vs non-preemptive
- Quantum too small → too many context switches; too big → FCFS

Common mistakes

× Mixing up turnaround and response time

Interview questions

☐ Compare round robin and SJF.

☐ What is starvation and how does ageing fix it?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · Gate Smashers (YouTube) (youtube.com)

Related: Processes and threads

L1 · CORE CORE CS

Deadlock ↑ Index

> **WHY IT MATTERS FOR YOU**
> Top-5 OS question. Also asked in DBMS (transaction deadlocks).

The concept

A set of processes each waiting for a resource held by another in the set, so none can proceed.

Coffman conditions, all four needed: mutual exclusion, hold and wait, no preemption, circular wait.

Handling: prevention (break one condition, e.g. always lock resources in a fixed global order to stop circular wait), avoidance (Banker's algorithm keeps the system in a safe state), detection + recovery (wait-for graph, kill or roll back), or ignore it (most OSes).

Key points

- Fixed lock ordering is the practical fix in code
- Livelock: processes keep changing state but make no progress
- Starvation ≠ deadlock

Common mistakes

× Listing only three conditions

Interview questions

☐ Four conditions of deadlock?
☐ How would you prevent deadlock in your code?
☐ Explain Banker's algorithm.

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · GfG: deadlock (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com)

Related: Synchronisation: mutex, semaphore, race conditions · Transactions, ACID and isolation

L2 · INTERVIEW-READY CORE CS

Synchronisation: mutex, semaphore, race conditions ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in OS rounds and C++/HFT interviews; producer-consumer is a classic whiteboard question.

The concept

A race condition happens when the result depends on the timing of threads touching shared data. The code that touches it is the critical section; it needs mutual exclusion, progress and bounded waiting.

A mutex is a lock with an owner: only the thread that locked it unlocks it. A semaphore is a counter: wait() decrements and blocks at 0, signal() increments. A binary semaphore looks like a mutex but has no owner; a counting semaphore guards N identical resources.

Producer-consumer with a bounded buffer: semaphores empty = N and full = 0, plus a mutex for the buffer. Condition variables let a thread sleep until a predicate becomes true (always wait in a loop).

Key points

- Spinlock: busy-waits, good for very short sections
- Atomics avoid locks for single variables
- Readers-writers and dining philosophers are the other classics

Code

```
std::mutex m; std::condition_variable cv; std::queue<int> q;
void produce(int x) { { std::lock_guard<std::mutex> g(m); q.push(x); } cv.notify_one(); }
int consume() { std::unique_lock<std::mutex> g(m); cv.wait(g, [] { return !q.empty(); });
    int x = q.front(); q.pop(); return x; }
```

Common mistakes

× Using if instead of while around a condition wait (spurious wake-ups)

Interview questions

- ☐ Mutex vs semaphore with an example.
- ☐ Write producer-consumer.
- ☐ What is a race condition?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · cppreference: thread support (en.cppreference.com)

Related: Deadlock · Processes and threads

L2 · INTERVIEW-READY CORE CS

Memory management and virtual memory ↑ Index

> **WHY IT MATTERS FOR YOU**
> Second most-asked OS area after processes. Also needed for binary exploitation (stack, heap, ASLR).

The concept

Each process sees its own virtual address space. The MMU translates virtual pages to physical frames using page tables; the TLB caches recent translations.

Why: isolation (one process can't read another's memory), programs bigger than RAM, simpler loading, and sharing (shared libraries, copy-on-write after fork).

A page fault happens when a page isn't in RAM: the OS loads it from disk and may evict another (FIFO, LRU, optimal; Belady's anomaly hits FIFO). Thrashing is when the system spends more time paging than running because working sets don't fit.

Key points

- Internal fragmentation (paging) vs external (segmentation)
- Multi-level page tables save memory
- Effective access time with TLB hit ratio

Common mistakes

× Confusing virtual memory with swap space only

Interview questions

☐ Why do we need virtual memory?
☐ What happens on a page fault?
☐ What is thrashing?
☐ LRU page replacement: implement it (→ LRU cache).

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · Gate Smashers (YouTube) (youtube.com)

Related: Processes and threads · Binary exploitation basics

L1 · CORE CORE CS

Normalisation and keys ↑ Index

> **WHY IT MATTERS FOR YOU**
> The standard DBMS opener in interviews and a frequent OA MCQ.

The concept

Keys: a super key identifies a row; a candidate key is a minimal super key; the primary key is the chosen candidate; a foreign key references another table's key.

Normalisation removes redundancy that causes insert, update and delete anomalies. 1NF: atomic values. 2NF: 1NF and no partial dependency on part of a composite key. 3NF: 2NF and no transitive dependency (non-key → non-key). BCNF: every determinant is a candidate key.

Denormalisation trades redundancy for faster reads, common in analytics and NoSQL.

Key points

- Example: Student(roll, name, dept, dept_head) → dept_head depends on dept → split into Student and Dept for 3NF
- Lossless join and dependency preservation
- ER diagram → tables

Common mistakes

× Not giving a concrete table example

Interview questions

☐ Explain 1NF to 3NF with one table.
☐ Difference between 3NF and BCNF?
☐ Primary vs unique key?

Learn from

GfG: normal forms (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com) · CMU 15-445 (free lectures) (15445.courses.cs.cmu.edu)

Related: Transactions, ACID and isolation · Indexing and B+ trees · SQL for interviews

L2 · INTERVIEW-READY CORE CS

Transactions, ACID and isolation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked at every product company, especially fintechs like Razorpay where money moves.

The concept

Atomicity (all or nothing, via undo logs), Consistency (constraints hold before and after), Isolation (concurrent transactions behave as if run in some order), Durability (committed data survives crashes, via write-ahead logging).

Anomalies: dirty read, non-repeatable read, phantom read. Isolation levels: Read Uncommitted (allows all), Read Committed (stops dirty reads), Repeatable Read (also stops non-repeatable reads), Serializable (stops phantoms).

Implementations use locks (two-phase locking) or MVCC (each transaction reads a snapshot; PostgreSQL, MySQL InnoDB).

Key points

- Read Committed prevents dirty reads
- 2PL: growing phase then shrinking phase
- Deadlocks between transactions are detected and one is rolled back

Common mistakes

× Saying 'Consistency' means the same as in CAP (it doesn't)

Interview questions

☐ Explain ACID with a bank transfer.
☐ Which isolation level prevents dirty reads?
☐ What is MVCC?

Learn from

GfG: ACID (geeksforgeeks.org) · CMU 15-445 (15445.courses.cs.cmu.edu)

Related: Normalisation and keys · Deadlock · CAP theorem and consistency

L2 · INTERVIEW-READY CORE CS

Indexing and B+ trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> 'Why is my query slow?' is a real-job question and a favourite follow-up in backend interviews.

The concept

An index is a separate structure that maps column values to row locations so the database avoids scanning the whole table.

Most databases use B+ trees: wide, shallow trees (each node holds hundreds of keys, sized to a disk page) with all values in linked leaves. A lookup touches about 3–4 pages for millions of rows, and range scans walk the leaves.

A clustered index stores rows in key order (one per table, usually the primary key); secondary indexes point to it. Composite indexes follow the leftmost-prefix rule.

Key points

- Indexes speed reads but slow writes and use space
- Covering index answers a query from the index alone
- Hash indexes: equality only, no ranges
- Use EXPLAIN to see the query plan

Common mistakes

- × Indexing low-cardinality columns (gender) expecting gains
- × Applying a function to an indexed column in WHERE stops index use

Interview questions

- ☐ How does an index speed up a query?
- ☐ Why B+ tree and not a binary tree or hash?
- ☐ Clustered vs non-clustered?

Learn from

Use The Index, Luke (free) `(use-the-index-luke.com)` · CMU 15-445 `(15445.courses.cs.cmu.edu)`

Related: Normalisation and keys · SQL for interviews

L1 · CORE CORE CS

SQL for interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> Many OAs include 1–2 SQL questions; analyst-flavoured and backend roles always do.

The concept

Order of evaluation: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. WHERE filters rows, HAVING filters groups.

Joins: INNER (matches only), LEFT (all left rows), self-join (employee–manager), anti-join (LEFT JOIN … WHERE right.id IS NULL).

Window functions compute over a partition without collapsing rows: ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD, running SUM() OVER (ORDER BY …).

Key points

- DENSE_RANK for 'Nth highest' with ties
- COUNT(col) ignores NULLs, COUNT(*) doesn't
- Parameterised queries prevent SQL injection

Code

```
SELECT dept, name, salary FROM (
  SELECT d.name AS dept, e.name, e.salary,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS r
  FROM employee e JOIN department d ON d.id = e.dept_id
) t WHERE r = 2;
```

Common mistakes

- × Using WHERE with an aggregate
- × Forgetting NULL handling in comparisons

Interview questions

- ☐ Second-highest salary per department.
- ☐ Find duplicate emails.
- ☐ Difference between RANK and DENSE_RANK?

Practise

LeetCode SQL 50 (leetcode.com) · SQLBolt (sqlbolt.com)

Learn from

SQLBolt (sqlbolt.com) · Mode SQL tutorial (mode.com)

Related: Indexing and B+ trees · SQL injection

L1 · CORE CORE CS

OSI and TCP/IP models ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first Computer Networks question in most interviews, and the vocabulary for everything in security.

The concept

OSI has 7 layers: Physical, Data link (MAC, switches), Network (IP, routers), Transport (TCP/UDP, ports), Session, Presentation, Application. TCP/IP merges them into 4: Link, Internet, Transport, Application.

Each layer wraps the one above with its own header (encapsulation): HTTP data → TCP segment → IP packet → Ethernet frame.

Key points

- Devices: hub (L1), switch (L2), router (L3)
- ARP maps IP → MAC
- Subnetting: /24 = 256 addresses, 254 usable hosts

Common mistakes

× Putting TLS or HTTP at the wrong layer without explaining

Interview questions

☐ Explain the OSI model.
☐ Which layer does a router work at?
☐ What is ARP?

Learn from

Gate Smashers (YouTube) (youtube.com) · Computer Networking: A Top-Down Approach (Kurose) (gaia.cs.umass.edu)

Related: TCP vs UDP and the handshake · What happens when you type a URL

L1 · CORE CORE CS

TCP vs UDP and the handshake ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in nearly every CN round; the 'why not 2-way?' follow-up catches people out.

The concept

TCP is connection-oriented and reliable: ordered delivery, retransmission, flow control (receiver window) and congestion control (slow start, congestion avoidance, fast retransmit). UDP is connectionless with no guarantees but low overhead and latency.

Handshake: client SYN (seq x) → server SYN-ACK (seq y, ack x+1) → client ACK (ack y+1). Three steps are needed so both sides confirm the other received their initial sequence number; two would leave the server unsure the client saw its SYN, and stale duplicate SYNs could open ghost connections.

Closing uses FIN/ACK in four steps; TIME_WAIT keeps the socket around to absorb late packets.

Key points

- Use UDP for video calls, games, DNS, live streams; QUIC builds reliability on UDP
- Flow control protects the receiver; congestion control protects the network
- SYN flood attack exploits half-open connections (SYN cookies defend)

Common mistakes

× Mixing flow and congestion control

Interview questions

☐ Why a 3-way and not 2-way handshake?
☐ When would you choose UDP?
☐ Flow vs congestion control?

Learn from

Kurose & Ross (gaia.cs.umass.edu) · Gate Smashers (YouTube) (youtube.com)

Related: OSI and TCP/IP models · HTTP, HTTPS and TLS

L2 · INTERVIEW-READY CORE CS

HTTP, HTTPS and TLS ↑ Index

> **WHY IT MATTERS FOR YOU**
> Bridges CN and security. Your AppSec and cloud work depends on it.

The concept

HTTP is a stateless request/response protocol: methods (GET, POST, PUT, DELETE, PATCH), status codes (2xx ok, 3xx redirect, 4xx client error, 5xx server error), headers, cookies for state.

HTTPS is HTTP over TLS. In TLS 1.3 the client and server agree keys with an ephemeral Diffie-Hellman exchange, the server proves its identity with a certificate signed by a trusted CA, and then traffic is encrypted with a fast symmetric cipher (AES-GCM or ChaCha20).

HTTP/2 multiplexes streams over one TCP connection; HTTP/3 runs over QUIC (UDP) to avoid head-of-line blocking.

Key points

- Asymmetric crypto for key exchange/auth, symmetric for bulk data
- Security headers: HSTS, CSP, X-Frame-Options
- Idempotent methods: GET, PUT, DELETE

Common mistakes

× Saying HTTPS encrypts with the server's public key directly (old RSA key exchange, not TLS 1.3)

Interview questions

☐ How does HTTPS work?
☐ Difference between HTTP/1.1, 2 and 3?
☐ What are cookies and how are they secured?

Learn from

The Illustrated TLS 1.3 Connection `(tls13.xargs.org)` · MDN HTTP docs `(developer.mozilla.org)`

Related: TCP vs UDP and the handshake · What happens when you type a URL · Hashing, encryption and passwords

L2 · INTERVIEW-READY CORE CS

What happens when you type a URL ↑ Index

> **WHY IT MATTERS FOR YOU**
> The single most popular CN interview question. It lets you show breadth from DNS to rendering.

The concept

1) Browser checks its caches and parses the URL. 2) DNS resolution: browser → OS → resolver → root → TLD (.com) → authoritative server, giving an IP. 3) TCP handshake (or QUIC). 4) TLS handshake. 5) HTTP request sent; it may pass through a CDN, load balancer and reverse proxy. 6) The server builds a response (app server, cache, database). 7) The browser parses HTML, fetches CSS/JS/images, builds the DOM and CSSOM, lays out and paints.

Go deeper on whichever layer the interviewer prods: DNS record types (A, AAAA, CNAME, MX), TTL caching, ARP on the local network, keep-alive.

Key points

- DNS uses UDP port 53 (TCP for large responses)
- Mention a CDN and load balancer to show system awareness
- Security add-on: HSTS, certificate validation

Common mistakes

- × Skipping DNS or TLS
- × Going so deep on one step you never finish

Interview questions

- ☐ What happens when you type google.com and press Enter?
- ☐ How does DNS resolution work?

Learn from

alex/what-happens-when (GitHub) (github.com)

Related: OSI and TCP/IP models · TCP vs UDP and the handshake · HTTP, HTTPS and TLS · Caching and CDNs

L1 · CORE CORE CS

OOP: the four pillars ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in every service-company interview and most product ones, often with 'show me in C++'.

The concept

Encapsulation: bundle data with methods and hide internals (private members, public interface). Abstraction: expose what an object does, not how (abstract classes, interfaces). Inheritance: a class reuses and extends another ('is-a'). Polymorphism: one interface, many forms; compile-time (overloading, templates) and run-time (virtual functions).

Prefer composition ('has-a') over deep inheritance trees.

Key points

- Overloading = same name, different parameters (compile time)
- Overriding = derived class redefines a virtual function (run time)
- Abstract class has at least one pure virtual function

Code

```
struct Shape { virtual double area() const = 0; virtual ~Shape() = default; };
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; } };
std::unique_ptr<Shape> s = std::make_unique<Circle>(2.0); // runtime polymorphism
```

Common mistakes

× Forgetting a virtual destructor in a base class

Interview questions

☐ Explain the four pillars with examples.
☐ Overloading vs overriding?
☐ Abstract class vs interface?

Learn from

learncpp.com: OOP chapters (learncpp.com) · GfG: OOP in C++ (geeksforgeeks.org)

Related: Virtual functions and the vtable · SOLID and design patterns · Low-level design (LLD)

L2 · INTERVIEW-READY CORE CS

Virtual functions and the vtable ↑ Index

> **WHY IT MATTERS FOR YOU**
> A C++-specific depth question that separates prepared candidates, and essential for HFT and binary exploitation.

The concept

Each class with virtual functions has a vtable: an array of function pointers to its versions of those functions. Each object stores a hidden vptr pointing to its class's vtable.

A virtual call loads the vptr, indexes the vtable and jumps: one extra indirection, and it usually blocks inlining (why low-latency C++ avoids virtual calls on hot paths, using templates/CRTP instead).

Constructors set the vptr, so calling a virtual function inside a constructor uses the current class's version, not the derived one.

Key points

- Virtual destructor needed for delete through a base pointer
- Object slicing: copying a derived object into a base by value
- Overwriting a vptr is a classic exploitation primitive

Common mistakes

× Thinking vtables are per object (they're per class)

Interview questions

☐ How does a virtual function call work?
☐ Why should base destructors be virtual?
☐ What is object slicing?

Learn from

learncpp: the virtual table (learncpp.com)

Related: OOP: the four pillars · Binary exploitation basics

L2 · INTERVIEW-READY CORE CS

SOLID and design patterns ↑ Index

> **WHY IT MATTERS FOR YOU**
> The vocabulary of every LLD round.

The concept

S: single responsibility. O: open for extension, closed for modification. L: Liskov substitution (a subclass must work wherever the base does). I: interface segregation (small, focused interfaces). D: dependency inversion (depend on abstractions, inject dependencies).

Patterns to know cold: Singleton (one instance; thread-safe via a function-local static in C++11), Factory (create objects without naming concrete classes), Strategy (swap algorithms at runtime), Observer (subscribers notified of events), plus Decorator and Adapter.

Key points

- Square-Rectangle is the classic Liskov violation
- Strategy = pass a pricing/parking-fee policy object
- Observer = notification systems

Common mistakes

× Naming patterns without a concrete use case

Interview questions

☐ Explain SOLID with examples.
☐ Implement a thread-safe singleton.
☐ Where would you use the strategy pattern?

Learn from

Refactoring.Guru design patterns (free) (refactoring.guru) · awesome-low-level-design (github.com)

Related: OOP: the four pillars · Low-level design (LLD)

L1 · CORE DESIGN

Low-level design (LLD) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Freshers at product companies get LLD more often than HLD. Flipkart, Swiggy, Atlassian and others run a machine-coding or LLD round.

The concept

Process: clarify requirements and scope (5 min) → list entities (nouns) and behaviours (verbs) → define classes, fields and relationships → key interfaces → walk through one flow → discuss extensions, concurrency and patterns used.

Practise: parking lot, library management, Splitwise, elevator, snake and ladder, BookMyShow, rate limiter, LRU cache, vending machine.

Key points

- Draw a class diagram first
- Use enums for states and types
- Mention thread safety where shared state exists
- Machine-coding rounds want runnable, clean code in 60–90 min

Common mistakes

× Starting to code before agreeing on requirements
× God classes that do everything

Interview questions

☐ Design a parking lot.
☐ Design Splitwise.
☐ Design an elevator system.

Learn from

awesome-low-level-design (github.com) · Refactoring.Guru (refactoring.guru)

Related: SOLID and design patterns · OOP: the four pillars · High-level system design (HLD)

L2 · INTERVIEW-READY DESIGN

High-level system design (HLD) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Big Tech asks light HLD even for new grads; a clear design discussion lifts you above candidates who only grind DSA. Your AWS SAA prep covers half of it.

The concept

Framework: requirements (functional + non-functional: scale, latency, availability) → back-of-envelope numbers (QPS, storage) → API → data model → high-level diagram → deep dive on the bottleneck → trade-offs.

Building blocks: load balancer, stateless app servers, cache (Redis), CDN, SQL vs NoSQL, replication, sharding, message queues (SQS/Kafka), object storage (S3), rate limiter, search index.

Starter designs: URL shortener, pastebin, rate limiter, chat app, news feed, notification service.

Key points

- 1 day ≈ 10^5 seconds; 1M requests/day ≈ 12 QPS
- Read-heavy → cache + replicas; write-heavy → sharding + queues
- State the trade-off you chose and why

Common mistakes

- × Jumping to microservices for a simple system
- × No numbers

Interview questions

- ☐ Design a URL shortener.
- ☐ How would you scale it to 100× traffic?

Learn from

System Design Primer (github.com) · roadmap.sh system design (roadmap.sh) · ByteByteGo blog (free posts) (blog.bytebytego.com)

Related: Caching and CDNs · CAP theorem and consistency · Load balancing, replication and sharding · Low-level design (LLD)

L2 · INTERVIEW-READY DESIGN

Caching and CDNs ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first scaling answer in any design question, and it links to LRU (a DSA favourite).

The concept

A cache keeps hot data in fast storage. Patterns: cache-aside (app reads cache, falls back to DB and fills cache), write-through (write cache and DB together), write-back (write cache, flush later).

Eviction: LRU, LFU, TTL. Invalidation is the hard part: set TTLs, delete on write, version keys.

A CDN caches static (and some dynamic) content at edge locations close to users (CloudFront on AWS).

Key points

- Cache stampede: many misses at once; use locking or early refresh
- Hit ratio is the metric
- LRU = hash map + doubly linked list

Common mistakes

× Caching data that must be strongly consistent without a plan

Interview questions

☐ How would you add caching to this system?

☐ How do you keep the cache consistent?

Learn from

System Design Primer: cache (github.com)

Related: High-level system design (HLD) · Linked lists

L2 · INTERVIEW-READY DESIGN

Load balancing, replication and sharding ↑ Index

> **WHY IT MATTERS FOR YOU**
> Core scaling vocabulary for HLD rounds and AWS SAA.

The concept

A load balancer spreads requests across servers (round robin, least connections, consistent hashing) and removes unhealthy ones via health checks. L4 balances by IP/port, L7 by HTTP content.

Replication copies data to more nodes for read scaling and availability (leader-follower; replication lag means followers can be stale). Sharding splits data across nodes by a key for write scaling; choose a key that spreads load evenly and avoids cross-shard queries.

Consistent hashing keeps most keys in place when nodes join or leave.

Key points

- AWS: ALB (L7), NLB (L4), Auto Scaling groups, RDS read replicas, DynamoDB partitions
- Hot shard problem: celebrity users

Common mistakes

× Sharding before trying caching and replicas

Interview questions

☐ How would you shard a users table?

☐ What is consistent hashing?

Learn from

System Design Primer (github.com)

Related: High-level system design (HLD) · CAP theorem and consistency · AWS core services and the SAA-C03 exam

L3 · ADVANCED DESIGN

CAP theorem and consistency ↑ Index

> **WHY IT MATTERS FOR YOU**
> An L3 depth question that shows you understand distributed trade-offs.

The concept

During a network partition, a distributed store must choose between consistency (every read sees the latest write) and availability (every request gets a response). PACELC adds: Else, when there's no partition, choose between latency and consistency.

Consistency models range from strong (linearisable) to eventual. Banking ledgers lean to consistency; social feeds lean to availability.

Key points

- Quorums: R + W > N gives read-your-writes
- Idempotency keys make retries safe (payments)

Common mistakes

× Saying a system can 'pick two' outside partitions

Interview questions

☐ Explain CAP with an example.

☐ How do you make a payment API safe to retry?

Learn from

Designing Data-Intensive Applications (book) (dataintensive.net)

Related: High-level system design (HLD) · Transactions, ACID and isolation

L1 · CORE SECURITY

OWASP Top 10 ↑ Index

> **WHY IT MATTERS FOR YOU**
> The shared language of AppSec. Even SDE interviewers are impressed when you name the risk in your own project.

The concept

The OWASP Top 10 (2021) lists the most critical web risks: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection (incl. XSS), A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 Server-Side Request Forgery.

Broken access control is #1: the app checks that you are logged in but not that you may access this object.

Key points

- Authentication = who you are; authorisation = what you may do
- Defence in depth: validate input, encode output, least privilege, logging
- A09 is exactly what TenantWall's CloudTrail + CloudWatch alarm addresses

Common mistakes

× Memorising the list without one exploit + one fix for each

Interview questions

☐ Name three OWASP risks and how you'd prevent them.

☐ Authentication vs authorisation?

Practise

PortSwigger Web Security Academy (portswigger.net)

Learn from

OWASP Top 10 (owasp.org) · OWASP Cheat Sheet Series (cheatsheetseries.owasp.org)

Related: SQL injection · Cross-site scripting (XSS) and CSRF · Broken access control and IDOR · Server-side request forgery (SSRF)

L1 · CORE SECURITY

SQL injection ↑ Index

> **WHY IT MATTERS FOR YOU**
> The classic vulnerability; first PortSwigger topic and a guaranteed security interview question.

The concept

Happens when user input is concatenated into a SQL query, so the input changes the query's structure. ' OR '1'='1 turns a login check into 'always true'; UNION SELECT reads other tables; blind SQLi infers data from true/false or timing differences.

Fix: parameterised queries / prepared statements, which send the query and the data separately so data can never become code. Add least-privilege DB accounts and input validation as extra layers. Escaping by hand is fragile.

Key points

- ORMs are safe unless you build raw queries with strings
- Second-order SQLi: stored input used later in a query

Example

```
// vulnerable
q = "SELECT * FROM users WHERE name = '" + name + "'";
// safe (prepared statement)
stmt = db.prepare("SELECT * FROM users WHERE name = ?"); stmt.bind(1, name);
```

Common mistakes

× Thinking input filtering alone fixes it

Interview questions

☐ What is SQL injection and how do you prevent it?
☐ Why do prepared statements work?

Practise

PortSwigger: SQL injection labs (portswigger.net)

Learn from

PortSwigger: SQL injection (portswigger.net) · OWASP SQLi prevention cheat sheet (cheatsheetseries.owasp.org)

Related: OWASP Top 10 · SQL for interviews

L1 · CORE SECURITY

Cross-site scripting (XSS) and CSRF ↑ Index

> **WHY IT MATTERS FOR YOU**
> The two most common client-side web bugs; asked in AppSec and full-stack interviews.

The concept

XSS: attacker-controlled script runs in another user's browser on your site. Reflected (from the request), stored (saved in the database), DOM-based (client JS writes untrusted data into the page). Impact: steal session data, act as the user.

Fix XSS: context-aware output encoding (HTML, attribute, JS, URL), frameworks that escape by default, a Content-Security-Policy, HttpOnly cookies.

CSRF: a malicious site makes the victim's browser send an authenticated request (e.g. change email). Fix: SameSite cookies, anti-CSRF tokens, re-authentication for sensitive actions.

Key points

- innerHTML with user data → DOM XSS
- SameSite=Lax is the modern default in browsers
- CORS is not a CSRF defence

Common mistakes

× Confusing XSS (runs script) with CSRF (forges a request)

Interview questions

☐ Difference between XSS and CSRF?
☐ How does CSP help?

Practise

PortSwigger: XSS labs (portswigger.net) · PortSwigger: CSRF labs (portswigger.net)

Learn from

PortSwigger: XSS (portswigger.net) · PortSwigger: CSRF (portswigger.net)

Related: OWASP Top 10 · HTTP, HTTPS and TLS

L2 · INTERVIEW-READY SECURITY

Broken access control and IDOR ↑ Index

> **WHY IT MATTERS FOR YOU**
> OWASP #1 and the most common bug-bounty finding. Directly related to TenantWall's tenant isolation.

The concept

IDOR (insecure direct object reference): the app uses an ID from the request (/invoice?id=1043) without checking the object belongs to the user. Change the ID, read someone else's data.

Other forms: missing function-level checks (normal user calls /admin API), privilege escalation via a role parameter, path traversal.

Fix: enforce authorisation on the server for every request and object (deny by default), centralise checks, use unguessable IDs only as extra defence, test with two accounts.

Key points

- Multi-tenant SaaS: every query filtered by tenant_id from the session, never from the request
- ABAC in TenantWall enforces this at the AWS layer with tags

Common mistakes

× Relying on hidden UI elements as access control

Interview questions

☐ What is IDOR and how would you test for it?

☐ How do you enforce tenant isolation?

Practise

PortSwigger: access control labs (portswigger.net)

Learn from

PortSwigger: access control (portswigger.net)

Related: OWASP Top 10 · ABAC: attribute-based access control · AWS IAM and least privilege

L2 · INTERVIEW-READY SECURITY

Server-side request forgery (SSRF) ↑ Index

> **WHY IT MATTERS FOR YOU**
> OWASP A10 and the bug behind major cloud breaches; ties web security to AWS.

The concept

The server fetches a URL the attacker controls (image preview, webhook, PDF generator). The attacker points it at internal addresses: localhost admin panels, internal services, or the cloud metadata endpoint (169.254.169.254) to steal temporary IAM credentials.

Fix: allow-lists of destinations, block private and link-local ranges after DNS resolution, disable redirects, and on AWS require IMDSv2 (session tokens) and give instances least-privilege roles.

Key points

- The 2019 Capital One breach involved SSRF + over-privileged IAM role
- flaws.cloud level 5 shows the metadata attack

Common mistakes

× Blocking by string matching (bypassed with decimal IPs, DNS rebinding)

Interview questions

☐ What is SSRF and why is it dangerous in the cloud?

Practise

PortSwigger: SSRF labs (portswigger.net) · flaws.cloud (flaws.cloud)

Learn from

PortSwigger: SSRF (portswigger.net)

Related: OWASP Top 10 · AWS IAM and least privilege · AWS core services and the SAA-C03 exam

L1 · CORE SECURITY

Hashing, encryption and passwords ↑ Index

> **WHY IT MATTERS FOR YOU**
> A common short question in security and backend interviews.

The concept

Hashing is one-way and fixed-length (SHA-256): used for integrity and fingerprints. Encryption is reversible with a key: symmetric (AES, one shared key, fast) or asymmetric (RSA/ECC, public/private key pair, used for key exchange and signatures).

Passwords: never encrypt, never plain-hash. Use a slow, salted password hash (Argon2id, bcrypt, scrypt). The salt stops precomputed rainbow tables; slowness stops brute force.

Digital signatures: hash the message, sign with the private key; anyone verifies with the public key.

Key points

- Encoding (Base64) is not encryption
- MAC/HMAC = integrity + authenticity with a shared key
- AES-GCM gives encryption + integrity

Common mistakes

× Saying 'we hash with MD5' for passwords

Interview questions

☐ Hashing vs encryption?
☐ How should passwords be stored?
☐ Symmetric vs asymmetric?

Learn from

OWASP Password Storage cheat sheet (cheatsheetseries.owasp.org) · CryptoHack (free) (cryptohack.org)

Related: HTTP, HTTPS and TLS · OWASP Top 10

L2 · INTERVIEW-READY SECURITY

Threat modelling (STRIDE) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Lets you add 'I threat-modelled it' to any project answer. Security-engineer interviews ask you to do it live.

The concept

Draw the system's data flow: users, processes, data stores, trust boundaries. For each element, ask STRIDE: Spoofing (identity), Tampering (data), Repudiation (no audit trail), Information disclosure, Denial of service, Elevation of privilege.

For each threat, record a mitigation and its priority. Four questions: what are we building, what can go wrong, what will we do about it, did we do a good job?

Key points

- Trust boundaries are where most bugs live
- Repudiation → logging (CloudTrail)
- Elevation of privilege → IAM least privilege

Common mistakes

× A list of threats with no mitigations

Interview questions

☐ Threat-model a login page.

☐ Threat-model your project.

Learn from

OWASP Threat Modeling (owasp.org) · Microsoft STRIDE (learn.microsoft.com)

Related: OWASP Top 10 · TenantWall (your research project)

L3 · ADVANCED SECURITY

Binary exploitation basics ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your L3 security rung and the binary taster from the roadmap. It makes your C/C++ and OS memory knowledge concrete, which HFT and systems interviewers value.

The concept

Stack layout: local buffers sit below the saved frame pointer and return address. Writing past a buffer (gets, strcpy without bounds) overwrites the return address and redirects execution.

Defences and bypasses: stack canaries (detect overwrite), NX/DEP (stack not executable → return-oriented programming chains existing code 'gadgets'), ASLR (randomised addresses → need an info leak), PIE, RELRO.

Other bug classes: format strings (printf(user)), use-after-free and double free on the heap, integer overflows. Tools: GDB with pwndbg, pwntools, Ghidra for reversing, checksec.

Key points

- Start with pwn.college's program-security modules
- Write a short write-up for every challenge you solve
- Secure C++: bounds-checked containers, avoid raw C strings

Common mistakes

× Jumping to heap exploitation before stack basics

Interview questions

☐ What is a buffer overflow and how do canaries and ASLR stop it?
☐ What is ROP?

Practise

pwn.college (pwn.college) · picoCTF (picoctf.org) · ROP Emporium (ropemporium.com)

Learn from

pwn.college (pwn.college) · Nightmare (free course) (guyinatuxedo.github.io)

Related: Memory management and virtual memory · Virtual functions and the vtable · CTFs and security labs

L2 · INTERVIEW-READY SECURITY

CTFs and security labs ↑ Index

> **WHY IT MATTERS FOR YOU**
> A proven hiring channel for security roles (see the KPMG story) and great resume proof.

The concept

Jeopardy CTFs have categories: web, pwn (binary exploitation), reverse engineering, crypto, forensics, misc. Each solved challenge gives a flag.

Path: OverTheWire Bandit (Linux) → picoCTF (all categories, beginner) → PortSwigger labs (web) → pwn.college (binary) → live CTFs on CTFtime with a team.

Write up each solve on a blog or GitHub: problem, approach, the key insight, the fix. Write-ups are what recruiters can read.

Key points

- Join or form a college team; India has many college CTFs
- Top-25% finish in a mid-rated CTF is a solid resume line

Common mistakes

× Only reading write-ups without solving

Interview questions

☐ Tell me about a CTF challenge you solved.

Practise

OverTheWire Bandit (overthewire.org) · picoCTF (picoctf.org) · CTFtime (ctftime.org) · TryHackMe (tryhackme.com)

Learn from

CTF101 (ctf101.org)

Related: Binary exploitation basics · Bug bounty and responsible disclosure

L4 · ELITE SECURITY

Bug bounty and responsible disclosure

↑ Index

> **WHY IT MATTERS FOR YOU**
> An accepted report is proof no college name can match. It's your L4 security goal.

The concept

Bug bounty programmes pay or credit researchers for reporting vulnerabilities within a defined scope. Read the scope and rules first; testing out of scope is illegal.

Start with recon on wide-scope programmes, focus on one bug class you know well (IDOR, access control, XSS), and read disclosed reports on HackerOne Hacktivity to learn what gets accepted.

Responsible disclosure: report privately, give details and impact, don't access more data than needed, wait for a fix before publishing.

Key points

- Quality report: steps to reproduce, impact, fix suggestion
- Indian govt sites: report to CERT-In
- Duplicates are normal at first

Common mistakes

× Running automated scanners against programmes that forbid them

Interview questions

☐ Walk me through a vulnerability you found.

Practise

HackerOne Hacktivity (hackerone.com) · Bugcrowd (bugcrowd.com)

Learn from

PortSwigger Academy (portswigger.net) · HackerOne Hacker101 (free) (hacker101.com)

Related: Broken access control and IDOR · CTFs and security labs

L1 · CORE CLOUD

AWS IAM and least privilege ↑ Index

> **WHY IT MATTERS FOR YOU**
> Week 1 of your AWS track and the heart of TenantWall. The most important AWS service for security roles.

The concept

Principals (users, roles) get permissions from JSON policies: Effect (Allow/Deny), Action, Resource, Condition. Roles give temporary credentials and are preferred over long-lived user keys.

Evaluation logic: everything starts as an implicit deny; an Allow grants access; an explicit Deny anywhere always wins (over any Allow). Permission boundaries and SCPs cap what can be allowed.

Tools from your professor's list: the IAM Policy Generator builds policy JSON; IAM Access Analyzer finds resources shared outside your account and validates policies; it can generate least-privilege policies from CloudTrail activity.

Key points

- Explicit deny > allow > implicit deny
- Never use the root account day to day; enable MFA
- Use roles for EC2/Lambda, not access keys

Example

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {"Bool": {"aws:SecureTransport": "false"}}
  }]
}
```

Common mistakes

× Policies with "Action": "*" and "Resource": "*"

Interview questions

☐ Explain IAM policy evaluation.
☐ Role vs user?
☐ How do you achieve least privilege?

Learn from

AWS docs: policy evaluation logic (docs.aws.amazon.com) · AWS docs: IAM Access Analyzer (docs.aws.amazon.com) · flaws2.cloud (flaws2.cloud)

Related: ABAC: attribute-based access control · TenantWall (your research project) · CloudTrail and CloudWatch alarms

L3 · ADVANCED CLOUD

ABAC: attribute-based access control ↑ Index

> **WHY IT MATTERS FOR YOU**
> The core idea of your TenantWall paper.

The concept

RBAC grants by role; ABAC grants by matching attributes. In AWS, attributes are tags on principals (aws:PrincipalTag/tenant) and on resources (aws:ResourceTag/tenant). One policy can say 'allow only when the principal's tenant tag equals the resource's tenant tag', so adding a tenant needs no new policy.

For S3, use object or bucket tags, or prefix-based conditions (s3:prefix with ${aws:PrincipalTag/tenant}). Protect the tags themselves: deny changing tenant tags (aws:TagKeys, aws:RequestTag) so a user can't re-tag into another tenant.

Key points

- Scales better than one role per tenant
- Pair with an explicit deny on tag modification
- Test isolation with two tenant users (should get AccessDenied)

Example

```
"Condition": {
  "StringEquals": {
    "aws:ResourceTag/tenant": "${aws:PrincipalTag/tenant}"
  }
}
```

Common mistakes

× Letting principals edit their own tags

Interview questions

☐ RBAC vs ABAC?

☐ How does TenantWall stop tenant A reading tenant B's data?

Learn from

AWS docs: What is ABAC? (docs.aws.amazon.com) · AWS docs: ABAC tutorial (docs.aws.amazon.com)

Related: AWS IAM and least privilege · TenantWall (your research project) · Broken access control and IDOR

L2 · INTERVIEW-READY CLOUD

CloudTrail and CloudWatch alarms ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your professor asked for exactly this in TenantWall: log every access attempt and alert on denied ones.

The concept

CloudTrail records API calls (who, what, when, from where). Management events are on by default; S3 object-level reads and writes need data events enabled on the trail. Send the trail to CloudWatch Logs.

Create a metric filter on the log group with the pattern { ($.errorCode = "*UnauthorizedOperation") || ($.errorCode = "AccessDenied*") }, which turns matching events into a metric. Put a CloudWatch alarm on it (e.g. ≥ 1 in 5 minutes) whose action publishes to an SNS topic with an email subscription.

Test: as tenant A, try to read tenant B's object; you should get AccessDenied, see the event in CloudTrail, the metric tick up and an email arrive.

Key points

- Confirm the SNS email subscription or nothing arrives
- Data events cost money; limit them to TenantWall buckets
- This maps to OWASP A09 (logging and monitoring failures)

Common mistakes

× Expecting S3 GetObject in CloudTrail without data events

Interview questions

☐ How would you detect unauthorised access attempts in AWS?

Learn from

AWS docs: CloudTrail data events (docs.aws.amazon.com) · AWS docs: CloudWatch alarms for CloudTrail (docs.aws.amazon.com)

Related: TenantWall (your research project) · AWS IAM and least privilege

L3 · ADVANCED CLOUD

TenantWall (your research project) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your strongest resume line: real cloud security research with a professor. It deserves a polished write-up, a diagram and a 2-minute interview story.

The concept

What: multi-tenant isolation on EC2 and S3 using ABAC (tag-matched IAM policies), so tenants can't reach each other's resources.

Your professor's additions: a CloudTrail trail with S3 data events; a CloudWatch metric filter on AccessDenied/UnauthorizedOperation with an alarm and SNS email; policies built with the Policy Generator and checked with IAM Access Analyzer; a flowchart and an AWS-icon architecture diagram; explicit deny statements (confirm the exact meaning with your professor).

Plan: deadline 21 Mar 2027. Build in Dec–Jan, run isolation tests and collect evidence (screenshots, alarm emails), write in Feb–Mar.

Key points

- Interview story: problem → design (ABAC) → how you proved isolation → what the alarm caught
- Diagram in diagrams.net with the AWS icon set
- Put the policies and test scripts on GitHub (no keys!)

Common mistakes

- × Committing AWS keys to GitHub
- × Leaving resources running and getting billed

Interview questions

- ☐ Tell me about your project.
- ☐ How did you verify isolation?
- ☐ What would you change at 1,000 tenants?

Learn from

diagrams.net (AWS icons) `(app.diagrams.net)` · AWS docs: ABAC `(docs.aws.amazon.com)`

Related: ABAC: attribute-based access control · CloudTrail and CloudWatch alarms · AWS IAM and least privilege · Threat modelling (STRIDE)

L2 · INTERVIEW-READY CLOUD

AWS core services and the SAA-C03 exam ↑ Index

> **WHY IT MATTERS FOR YOU**
> 30 minutes a day from 16 Nov, exam in the week of 19 Apr 2027. Cloud skills plus a cert lift both SDE and security profiles.

The concept

Compute: EC2 (instances, AMIs, security groups), Auto Scaling, Lambda. Storage: S3 (classes, versioning, lifecycle, encryption, Block Public Access), EBS, EFS. Databases: RDS (Multi-AZ for HA, read replicas for scale), DynamoDB, ElastiCache. Networking: VPC, public/private subnets, route tables, internet and NAT gateways, ALB/NLB, Route 53, CloudFront. Integration: SQS, SNS. Security: IAM, KMS, CloudTrail, GuardDuty.

Exam domains: secure architectures 30%, resilient 26%, high-performing 24%, cost-optimised 20%. 65 questions, 130 minutes, pass mark 720/1000.

Key points

- Multi-AZ = availability; read replica = read scale
- NAT gateway lets private subnets reach the internet outbound only
- Set a billing alarm in week 1

Common mistakes

- × Only watching videos; do the labs
- × Leaving NAT gateways running (they cost hourly)

Interview questions

- ☐ Design a highly available web app on AWS.
- ☐ Security group vs NACL?

Learn from

AWS SAA-C03 exam guide (aws.amazon.com) · freeCodeCamp SAA course (YouTube) (youtube.com) · AWS Skill Builder (free tier) (skillbuilder.aws)

Related: AWS IAM and least privilege · Load balancing, replication and sharding · High-level system design (HLD)

L2 · INTERVIEW-READY AI

LLMs and RAG apps ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your AI track from 14 Dec (≤1 h). An AI + security project is a rare combination on a fresher resume.

The concept

An LLM predicts the next token from a prompt; it knows only its training data and what you put in the context window.

RAG (retrieval-augmented generation): split documents into chunks, embed them as vectors, store them in a vector index; at question time, embed the question, retrieve the closest chunks and put them in the prompt so the answer is grounded in your data.

Tool calling lets the model request a function (search, database query) that your code runs. Evals: a fixed set of questions with expected answers to measure changes.

Key points

- Chunk size and overlap matter more than the model
- Always cite retrieved sources in answers
- Evaluate before and after every change

Common mistakes

- × No evaluation set
- × Putting secrets in prompts

Interview questions

- ☐ How does RAG work?
- ☐ How would you evaluate an LLM feature?

Learn from

DeepLearning.AI short courses (free) (deeplearning.ai) · Anthropic docs (docs.anthropic.com)

Related: Prompt injection and AI security

L3 · ADVANCED AI

Prompt injection and AI security ↑ Index

> **WHY IT MATTERS FOR YOU**
> Where your AI and cyber tracks meet. Few freshers can talk about it.

The concept

Prompt injection: untrusted text (a web page, an email, a document in your RAG store) contains instructions that the model follows instead of the developer's. Direct injection comes from the user; indirect injection hides in retrieved content.

Risks grow with tools: an injected instruction can make an agent leak data or take actions. Defences: treat model output as untrusted, least-privilege tools, human confirmation for risky actions, separate trusted instructions from data, output filtering, and red-team tests.

The OWASP Top 10 for LLM applications lists prompt injection, sensitive information disclosure, supply chain, data poisoning, improper output handling, excessive agency and more.

Key points

- Build a test set of injection attempts for your RAG project
- Log and monitor model actions

Common mistakes

× Believing a system prompt alone stops injection

Interview questions

☐ What is prompt injection and how would you mitigate it?

Learn from

OWASP LLM Top 10 (genai.owasp.org) · Gandalf (Lakera) prompt-injection game (gandalf.lakera.ai)

Related: LLMs and RAG apps · OWASP Top 10

L3 · ADVANCED HFT

Probability for HFT interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> The later HFT track (Apr–Jul 2027). It also makes maths-heavy OA questions easy.

The concept

Counting: permutations, combinations, stars and bars, inclusion–exclusion. Conditional probability and Bayes' rule: $P(A|B) = P(B|A) \cdot P(A) / P(B)$.

Expectation is linear even for dependent variables: $E[X + Y] = E[X] + E[Y]$. Indicator variables turn counting questions into sums of probabilities (expected number of fixed points in a random permutation = 1).

Condition on the first step for recursive questions: expected flips to get HH satisfies a small system of equations (answer 6). Gambler's ruin and random walks follow the same idea.

Key points

- Zetamac mental arithmetic: 40+ then 50+
- Fair price of a game = its expected value
- Write every solved puzzle's key idea in one line

Common mistakes

× Assuming independence without checking

Interview questions

☐ Expected number of rolls to see a 6?
☐ You flip until HH; expected flips?
☐ Bayes: a test is 99% accurate for a 1% disease; P(sick | positive)?

Practise

Brainstellar (brainstellar.com) · Zetamac (arithmetic.zetamac.com)

Learn from

Harvard Stat 110 (projects.iq.harvard.edu) · Blitzstein & Hwang (free book) (projects.iq.harvard.edu)

Related: Competitive programming and contests

L1 · CORE CAREER

The interview loop (UMPIRE) ↑ Index

> **WHY IT MATTERS FOR YOU**
> A fixed routine for every DSA interview so nerves don't decide your structure.

The concept

Understand: restate the problem, ask about input size, ranges, duplicates, empty input; agree on 2 examples. Match: name the pattern. Plan: describe the approach in words and give its complexity; get a nod. Implement: clean code, good names. Review: dry-run an example and an edge case. Evaluate: complexity, trade-offs, what you'd improve.

Always say the brute force first. It shows you can solve it, gives you a fallback and often reveals the optimisation.

Key points

- Spend ~5 min understanding, ~5 planning, ~20 coding, ~5 testing in a 45-min round
- If stuck, think aloud about what you know; interviewers give hints to people who talk

Common mistakes

× Coding before the interviewer agrees with the approach

Learn from

CodePath UMPIRE guide (guides.codepath.org)

Related: Speaking while you code · Patterns, not problems · Mock interviews

L1 · CORE CAREER

Online assessments (OAs) ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first filter for almost every off-campus role. Many strong candidates fail here from poor tactics, not poor skill.

The concept

Typical format: 2–3 DSA problems in 60–90 minutes on HackerRank, HackerEarth, CodeSignal or Mettl, sometimes with MCQs on CS fundamentals and aptitude.

Tactics: read all problems in the first 2 minutes; solve in order of confidence; submit a brute force for partial test cases if the optimal idea isn't coming; check overflow (long long), empty and max-size inputs; stop optimising in the last 10 minutes.

Practise in the same environment: do timed sets on HackerRank, not only LeetCode, because I/O handling differs (you often write main() and read input yourself).

Key points

- Keep your C++ template ready to paste
- Proctoring: webcam, no tab switching
- Simulate a full 90-minute OA weekly from June

Common mistakes

× Spending 60 minutes on problem 1
× Forgetting to read input in the given format

Learn from

HackerRank interview prep kit (hackerrank.com)

Related: Competitive programming and contests · The error log · Time and space complexity

L2 · INTERVIEW-READY CAREER

Mock interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviewing is a separate skill from solving. Mocks are where you find out what breaks under pressure.

The concept

From 31 May, two mocks a week: one peer DSA mock and one Core CS / design mock. Rotate roles: being the interviewer teaches you what they look for.

Free options: Pramp (peer matching), friends from college or Discord groups, the AI mock-interviewer prompt. Record yourself when practising alone.

After each mock, write three things: what went well, what broke, one fix for next time.

Key points

- Score yourself 1–5 on problem solving, code, communication
- Do mocks at the same time of day as real interviews

Common mistakes

× Only doing mocks with friends who go easy on you

Learn from

Pramp (pramp.com) · interviewing.io (free practice when available) (interviewing.io)

Related: The interview loop (UMPIRE) · Speaking while you code · Behavioural interviews and STAR stories

L1 · CORE CAREER

Behavioural interviews and STAR stories ↑ Index

> **WHY IT MATTERS FOR YOU**
> Amazon's bar raiser and every HR round use behavioural questions. Six good stories cover almost all of them.

The concept

STAR: Situation (one line of context), Task (your responsibility), Action (what you did — most of the answer, in 'I' not 'we'), Result (outcome with a number, plus what you learned).

Prepare six stories that each fit several questions: a hard technical problem (TenantWall IAM), a failure, a conflict, learning something fast (C++/AWS), going beyond the task (a security finding), helping or leading someone.

For Amazon, map each story to Leadership Principles: Ownership, Dive Deep, Learn and Be Curious, Bias for Action, Deliver Results, Customer Obsession.

Key points

- Keep each story to ~2 minutes
- Have a 60-second 'tell me about yourself'
- Real numbers beat adjectives

Common mistakes

× Talking about the team instead of your actions
× No result or lesson

Interview questions

☐ Tell me about a time you failed.
☐ Tell me about a conflict with a teammate.
☐ Why this company?

Learn from

Amazon Leadership Principles (amazon.jobs)

Related: Mock interviews · TenantWall (your research project)

L1 · CORE CAREER

The one-page resume ↑ Index

> **WHY IT MATTERS FOR YOU**
> Off-campus, the resume is read for 6–10 seconds. It decides whether any of your work gets seen.

The concept

One page, single column, no photos, tables or graphics (ATS parsers break on them), exported as PDF named Firstname_Lastname_SDE.pdf.

Order for you: contact links (LinkedIn, GitHub, LeetCode/Codeforces) → education (CGPA if ≥ 7.5) → experience → projects (TenantWall first) → achievements (ratings, CTF ranks, AWS SAA, paper) → skills.

Bullet formula: action verb + what you built + technology + measurable result. 'Built X using Y, which achieved Z.'

Keep a 2-page private master resume with every bullet; cut 3 one-page versions (SDE, Security/AppSec, Cloud) and tailor each application in 5 minutes.

Key points

- Numbers in every project bullet
- Link a live demo or video
- Have two seniors review it

Common mistakes

× Listing every technology you've touched
× Tutorial projects (to-do app, weather app)

Learn from

Overleaf resume templates (Jake's resume) (overleaf.com)

Related: Projects that get shortlisted · Referrals and LinkedIn outreach

L2 · INTERVIEW-READY CAREER

Projects that get shortlisted ↑ Index

> **WHY IT MATTERS FOR YOU**
> Two strong projects with numbers beat six tutorials, and give you the stories for interviews.

The concept

Every project needs: a live link or demo video, a README with an architecture diagram, at least one number (latency, requests/s, users, bugs found), and a 'what I'd do next' line.

For your profile: TenantWall (cloud security research), a rate-limited URL shortener on AWS with load-test numbers (backend + design), a log-based intrusion detector (security + data), a secure RAG assistant with prompt-injection tests (AI + AppSec), and 3+ merged open-source PRs.

Threat-model each project and fix one issue; it becomes a unique interview story.

Key points

- Pin them on GitHub with clean READMEs
- Write tests for core logic
- Never commit secrets (.env in .gitignore)

Common mistakes

× Unfinished projects with no README

Interview questions

☐ Walk me through your project's architecture.
☐ What was the hardest bug?
☐ How would it scale?

Learn from

roadmap.sh project ideas (roadmap.sh)

Related: TenantWall (your research project) · The one-page resume · Threat modelling (STRIDE)

L1 · CORE CAREER

Referrals and LinkedIn outreach ↑ Index

> **WHY IT MATTERS FOR YOU**
> The strongest off-campus channel: the Eternal (Zomato) story in this guide came through a referral.

The concept

Find employees or alumni at the target company on LinkedIn (search 'company + RAIT' first, then any engineer). Send a short, specific note with the job ID and your resume link. Ask for a referral, not 'a job'.

Make it easy to say yes: one-line intro, two proofs (problems solved, project), the exact role and job ID, a resume link. Thank them either way and follow up once after 5–7 days.

Build the network before you need it: comment thoughtfully on engineers' posts, share your project write-ups.

Key points

- 10 referral messages a week
- Alumni reply far more often than strangers
- Track every message in your sheet

Example

```
Hi [Name], I'm a 3rd-year CSE (Cybersecurity) student; I've solved 450+ DSA problems and built [project in one line].
I'm applying for [Role, Job ID] at [Company]. Would you be open to referring me?
Resume: [link]. Thank you!
```

Common mistakes

- × 'Hi, any openings?' messages with no context
- × Messaging the same person repeatedly

Related: The application pipeline · The one-page resume · Cold emailing startups

L2 · INTERVIEW-READY CAREER

Cold emailing startups ↑ Index

> **WHY IT MATTERS FOR YOU**
> Startups hire interns without formal drives. A specific email to a founder can skip the whole funnel.

The concept

Target funded Indian startups (Inc42, Entrackr funding news, YC India companies). Email the founder or CTO directly.

Five lines: you use or studied their product; who you are; one relevant thing you built (with a number); one specific observation (a bug, a missing security header, a feature idea); the ask (internship for N months, 15-minute call). Attach or link resume and GitHub.

Send 5 a week, personalised. A security observation done responsibly (no exploitation) is a strong opener for a cyber student.

Key points

- Subject line: role + your hook
- Follow up once after a week

Common mistakes

- × Mass-mailing the same template
- × Testing their systems without permission

Learn from

Inc42 funding news (inc42.com) · Y Combinator companies (India filter) (ycombinator.com)

Related: Referrals and LinkedIn outreach · The application pipeline

L1 · CORE CAREER

The application pipeline ↑ Index

> **WHY IT MATTERS FOR YOU**
> Off-campus hiring is a numbers game. A batched system makes 50 good applications a month take 90 minutes a week.

The concept

Collect: saved job alerts on LinkedIn and Naukri, 40 bookmarked careers pages checked Monday and Thursday, Unstop and HackerEarth for hiring contests.

Apply in two 45-minute batches (Mon, Thu): tailor the resume in 5 minutes, apply within 48 hours of posting, send the referral message for each job.

Track everything in one sheet: company, role, date, channel, contact, status, next action. Follow up once after 5–7 days. Expect a 2–5% reply rate; volume plus quality produces interviews.

Key points

- Apply even before you feel ready (≈150 problems + 1 project)
- Hiring contests skip resume filters
- Re-use 'why this company' answers

Common mistakes

- × Applying only on campus
- × No tracker, so no follow-ups

Learn from

Unstop (unstop.com) · LinkedIn Jobs (linkedin.com)

Related: Referrals and LinkedIn outreach · Cold emailing startups · The one-page resume

L2 · INTERVIEW-READY CAREER

Preparing for a specific company ↑ Index

> **WHY IT MATTERS FOR YOU**
> Once you have an interview, one focused week beats general practice.

The concept

1) Read 5–10 recent interview experiences for that company and role on LeetCode Discuss and GeeksforGeeks; note round types and repeated questions. 2) Solve its most frequent recent LeetCode-tagged problems (top 30). 3) Map its values to your STAR stories. 4) Use the product: find one improvement and one security thought to mention.

Ask the recruiter about the round structure; they usually tell you.

Key points

- LeetCode company tags need Premium; free alternatives are Discuss posts and GitHub lists of company questions
- Glassdoor interview section

Common mistakes

× Only solving tagged problems without fundamentals

Learn from

LeetCode Discuss: interview experiences (leetcode.com) · GfG interview experiences (geeksforgeeks.org)

Related: Behavioural interviews and STAR stories · Mock interviews

L2 · INTERVIEW-READY CAREER

Offers, CTC and negotiation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Freshers often lose money or sign bad bonds by not knowing what to ask.

The concept

CTC (cost to company) includes fixed salary, variable/bonus, joining bonus, stock (RSUs vesting over years), benefits. In-hand monthly pay is fixed salary minus tax and PF, divided by 12. Ask for the breakdown in writing.

Check the service agreement or bond clause, notice period, location and relocation support before signing.

Negotiation: research ranges on levels.fyi and Glassdoor; avoid naming a number first; if you have a competing written offer, mention it politely. Mass-recruiter fresher offers are rarely negotiable; product companies sometimes adjust joining bonus.

Key points

- RSUs: 4-year vesting is common, often with a 1-year cliff
- PPO = pre-placement offer from an internship

Common mistakes

× Comparing CTCs without separating fixed and variable

Interview questions

☐ What are your salary expectations?

Learn from

levels.fyi (levels.fyi) · Glassdoor India (glassdoor.co.in)

Related: The application pipeline

L3 · ADVANCED CAREER

Open source and GSoC ↑ Index

> **WHY IT MATTERS FOR YOU**
> A brand that removes the college question (see the tier-3 MCA student's GSoC story), and real code review experience.

The concept

Pick one organisation whose tech you use (C++, Python, security tools). Set up the project locally, read the contribution guide, start with 'good first issue' labels, and join their chat.

Aim for 3+ merged PRs before applying to programmes. GSoC contributor applications usually open in March–April (check the timeline each year); LFX Mentorship runs several cohorts a year; MLH Fellowship is another route.

Your proposal should show you already understand the codebase: link your merged PRs and give a week-by-week plan.

Key points

- Quality of PRs > quantity
- Maintainers often refer good contributors for jobs

Common mistakes

- × Typo-fix PRs to many repos
- × Starting a month before the deadline

Learn from

Google Summer of Code (summerofcode.withgoogle.com) · LFX Mentorship (mentorship.lfx.linuxfoundation.org) · MLH Fellowship (fellowship.mlh.io)

Related: Projects that get shortlisted